



分布式系統安全

李琦

清華大學網研院



本章的內容組織



第一節

建立安全、穩定的交互網絡

- 交互網絡的組成
- 交互信道安全
- 彈性覆蓋網絡
- 防禦日蝕攻擊

瞭解交互網絡的組成以及如何
如何構建安全的交互網絡



第二節

分布式算法

- 協作過程中的安全隱患
- 時鐘同步算法
- 并發問題
- 故障容錯的一致性算法

挖掘分布式系統各組件之間如何
如何進行協同



第三節

信任問題

- 訪問控制
- 信任評價模型
- 拜占庭容錯

瞭解分布式系統中的信任問題
及解決方案

為分布式系統各組件之間的協
同提供交互基礎

為分布式系統添加信任面，實現
可信協同



第1節 建立安全、穩定的交互網絡

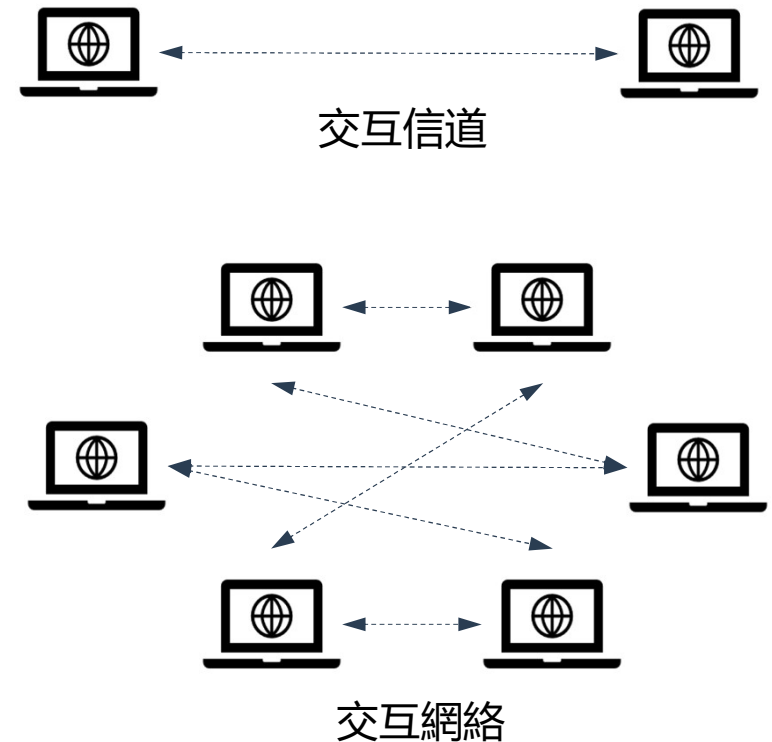
- ✓ 交互網絡的組成
- ✓ 交互信道安全
- ✓ 彈性覆蓋網絡
- ✓ 防禦日蝕攻擊



交互網絡的組成

在分布式系統中，節點之間可以通過交互信道實現消息傳遞；分布式節點選擇鄰居節點并建立交互信道，從而構成一個交互網絡

- **交互信道：**兩個主機通過交互信道實現host-to-host的通信，常見的交互信道有UDP、TCP和TLS等
- **鄰居選擇機制：**每個節點依靠鄰居選擇機制選取系統中的部分或全部節點建立交互信道，從而建立鄰居關係，節點可通過鄰居節點與其它非鄰居節點交互

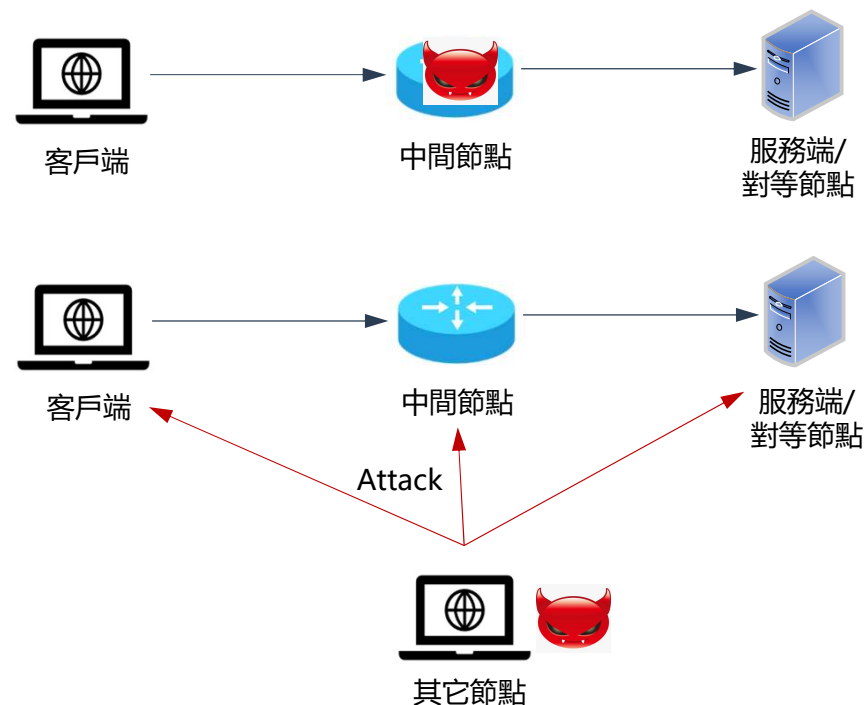




交互信道中的安全隱患

- 根據網絡體系結構的不同層面進行分類
 - 網絡層面：網絡中傳輸的數據包可能出現**丟失**、**重複**和**泄漏**
 - 應用層面：節點間的交易請求可能出現**泄漏**、**重放**、**篡改**和**偽造**

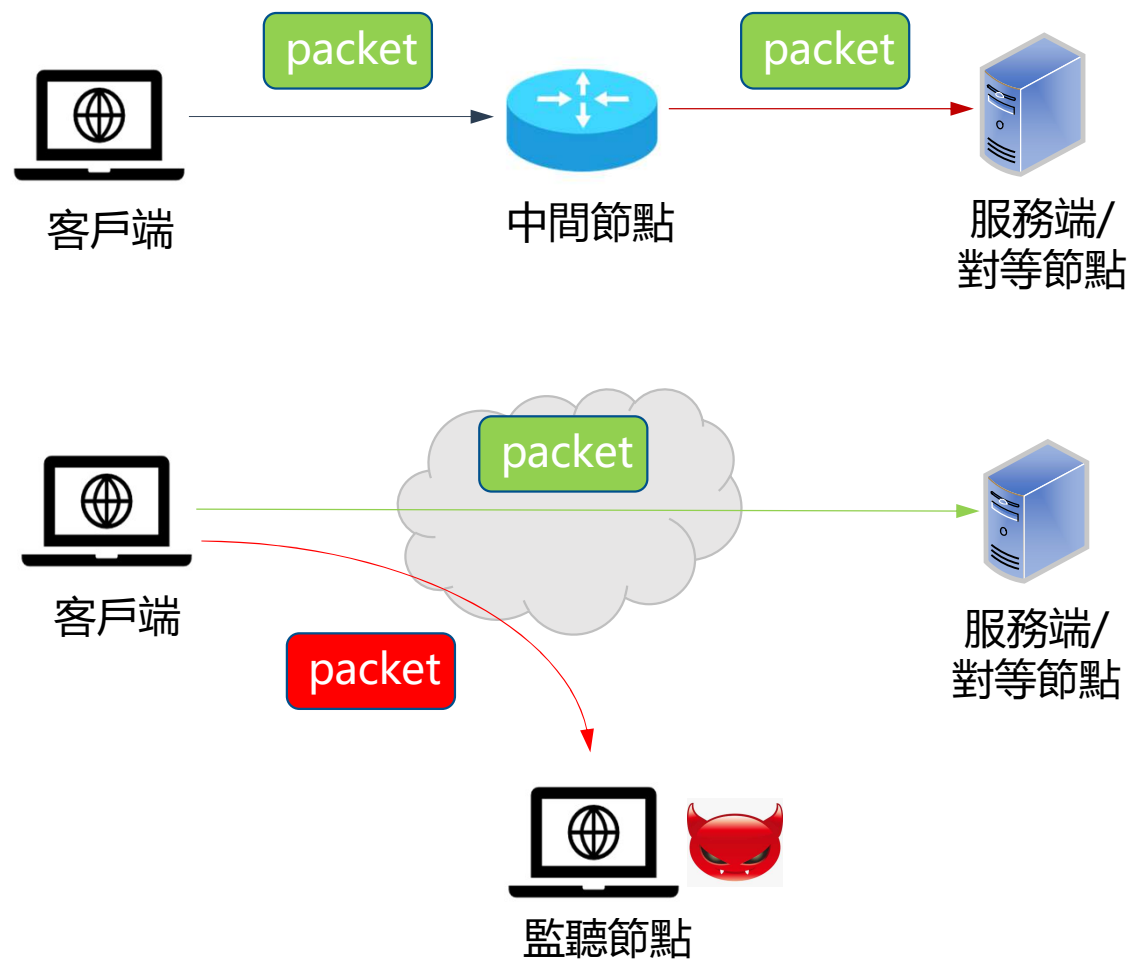
- 根據攻擊者的位置分類
 - **中間轉發節點**：直接參與轉發數據包，攻擊能力強，可以監聽、篡改、偽造、重放、延遲甚至丟棄轉發信息
 - **其它節點**：攻擊能力較弱，但能夠通過DDoS、中間人等攻擊影響交互信道





交互信道中的安全隱患

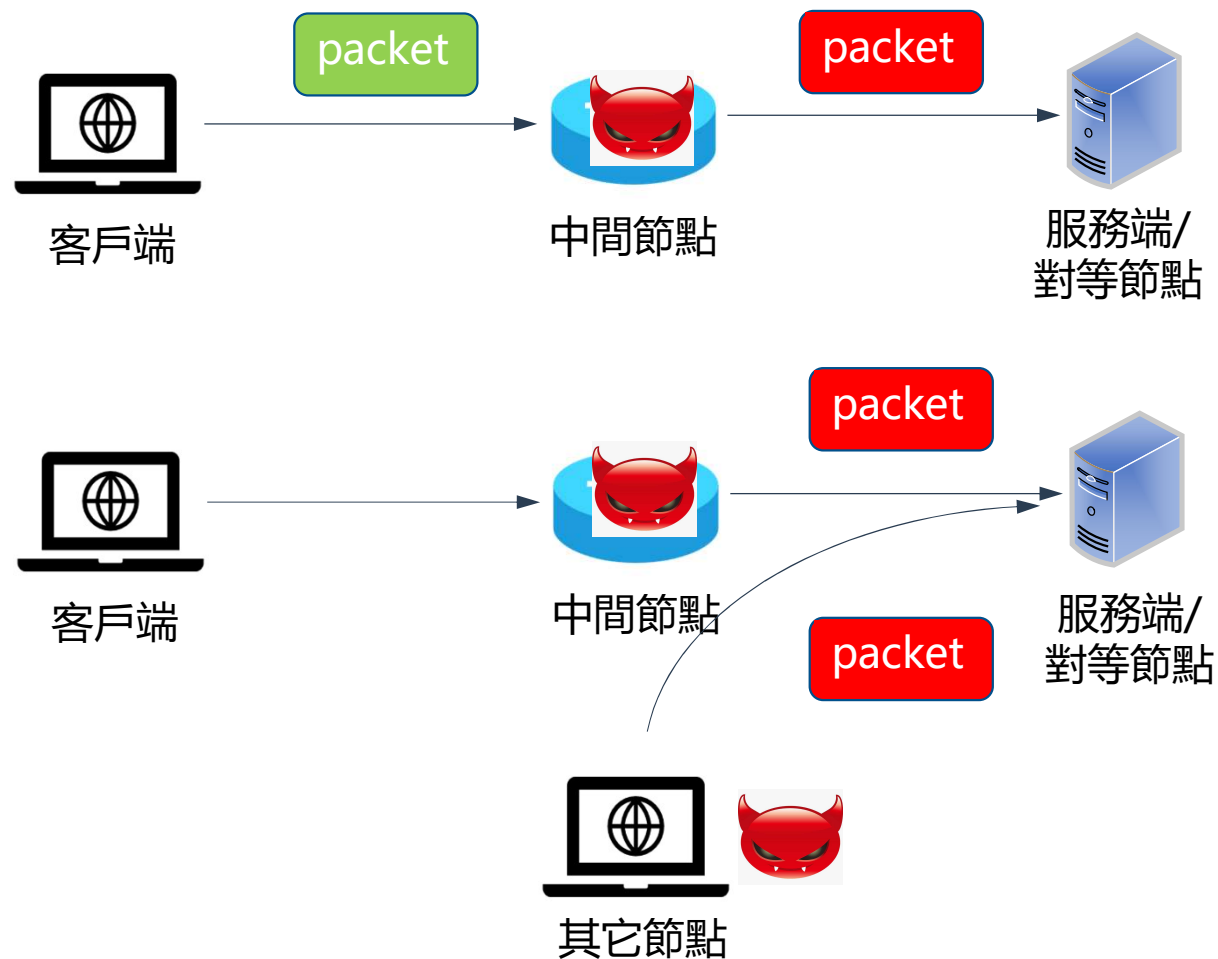
- **數據包丟失和重複：**當網絡出現擁塞或被攻擊時，客戶端發送的數據包可能會丟失；此時客戶端重傳數據包可能導致服務端收到重複數據包
- **信息泄漏：**明文傳輸數據包時可能會發生，節點可以將監聽到的數據包進行重組，從而獲取應用層數據信息





交互信道中的安全隱患

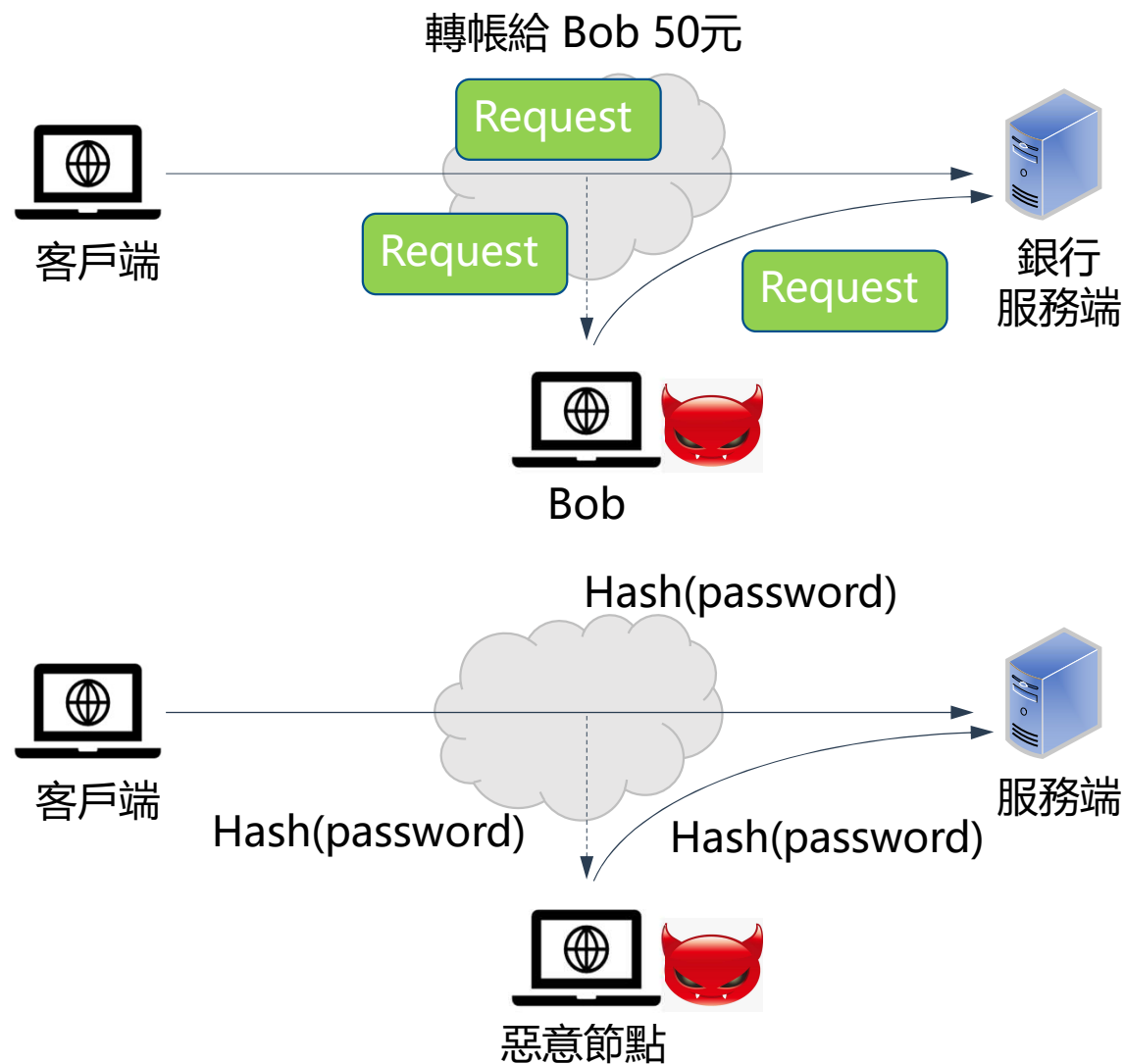
- 數據包篡改
 - 中間轉發節點可能篡改經過的數據包
 - 其它節點偽造
- 數據包偽造
 - 中間轉發節點或其它節點偽造數據包
 - 其它節點偽造客戶端發送應用層請求





交互信道中的安全隱患

- 重放攻擊：惡意節點通過監聽數據包構造客戶端發往服務端的應用層請求，重新將該請求多次發送給服務端
- 客戶端發送交易轉帳給Bob，Bob監聽到該交易後進行重放，從而完成多次轉帳
- 客戶端發送密碼登入服務端，惡意節點監聽到消息後進行重放，從而冒充客戶端登入服務端

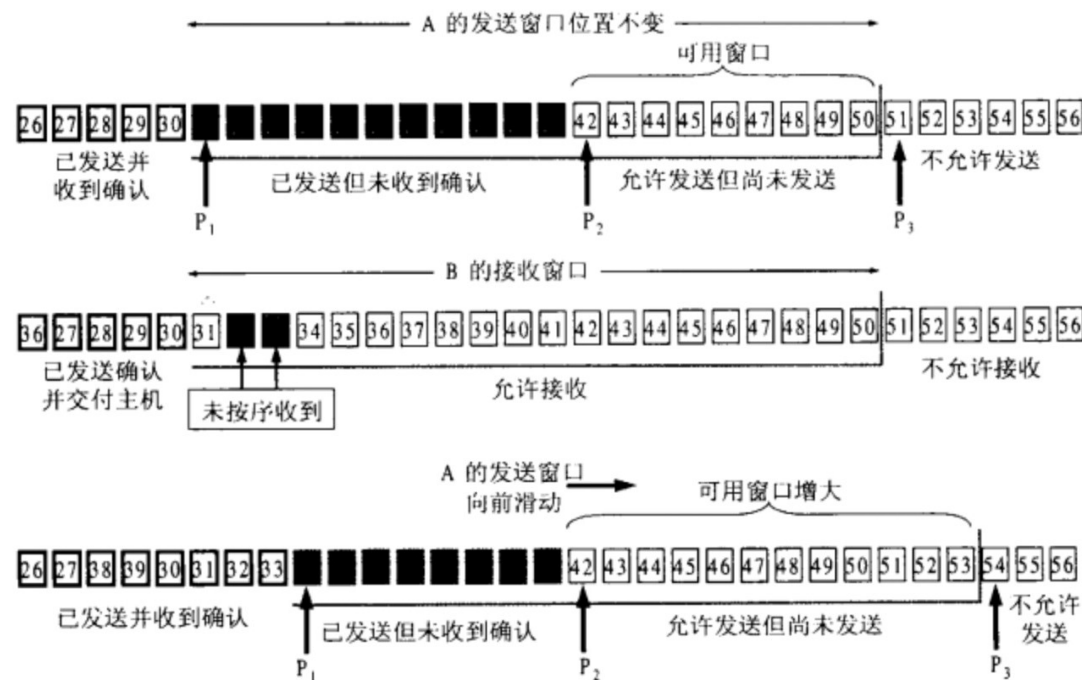




构建TCP交互信道

TCP協議采用滑動窗口協議確保數據包有序提交，并解决重複問題；采用超時重傳機制解决鏈路擁塞導致的數據包丢失問題

- 當未按序接收時，等待前面的數據包完成接收後再將數據包序列提交到上層應用
- 當長時間未收到數據包時，觸發超時重傳機制，請求源端超時重傳數據包



注. 圖片來自來源: 卿萃科技ALIFPGA
<https://www.cnblogs.com/alifpga/p/7675850.html>

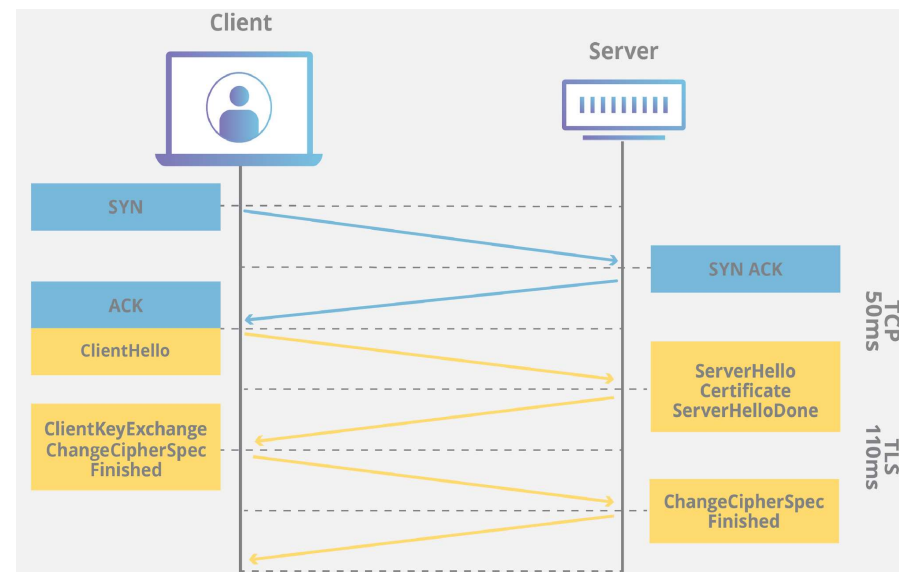


構建TLS交互信道

TLS 協議建立在TCP協議之上運行，提供以下三個安全功能：

- **數據完整性：**驗證數據是否偽造而來或未遭篡改過
- **身份認證：**獲取通信方的身份，為訪問控制機制作支撐
- **數據加密：**協商會話密鑰用于數據加密，防止第三方竊聽傳輸數據

- 基于PKI獲取對方公鑰證書
- 基于公鑰非對稱加密協商隨機數
- 基于協商的隨機數生成會話密鑰，從而建立安全信道
- 基于會話密鑰對傳輸數據加密，並生成消息認證碼對消息完整性進行驗證





TLS防重放攻擊

- TLS交互信道帶來的優勢
 - 基于TCP協議確保傳輸層面數據包不會出現重複
 - 通過數據加密確保惡意節點無法通過監聽獲取應用層消息

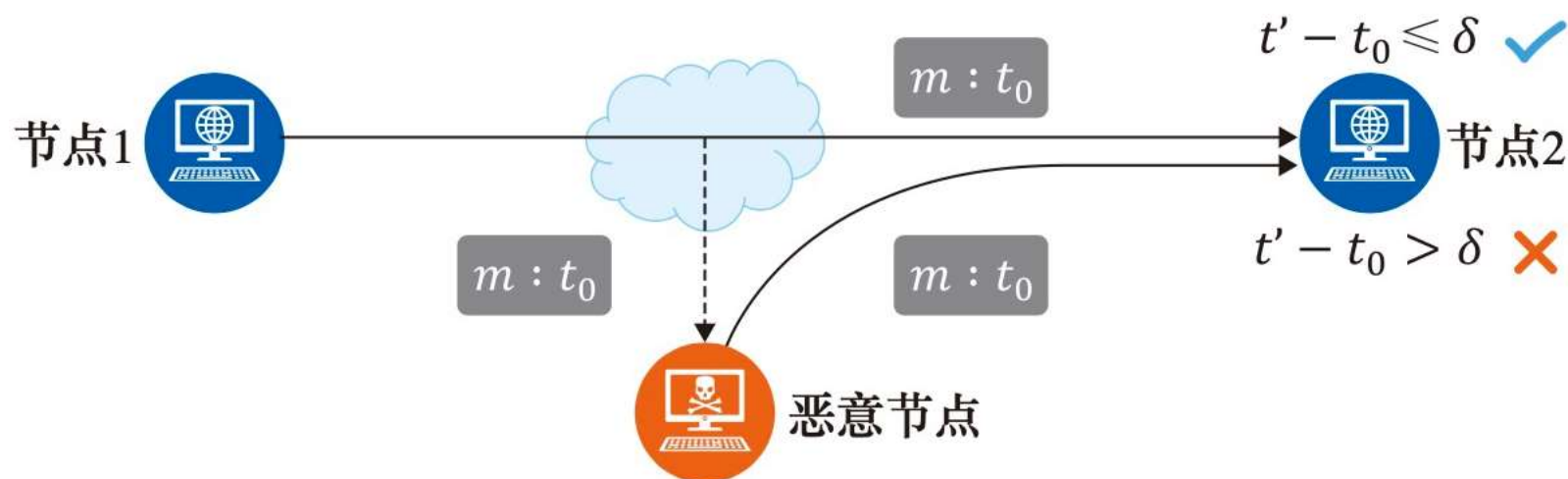
問題：TLS是否可以完全解決重放攻擊？

- 以下消息依然可能遭受重放攻擊
 - 客戶端發起的TLS請求建立信息ClientHello
 - TLS1.3爲了加速握手過程而引入的0-RTT信息



基于時間戳防禦重放攻擊

- 交互雙方完成精確時鐘同步
- 節點1在發送的消息上附帶本地時間戳 t_0
- 節點2收到消息後，用本地時間減去消息中所附帶的時間戳得到一個時間差
- 若時間差小于允許值 δ ，則接受該消息；否則丟棄



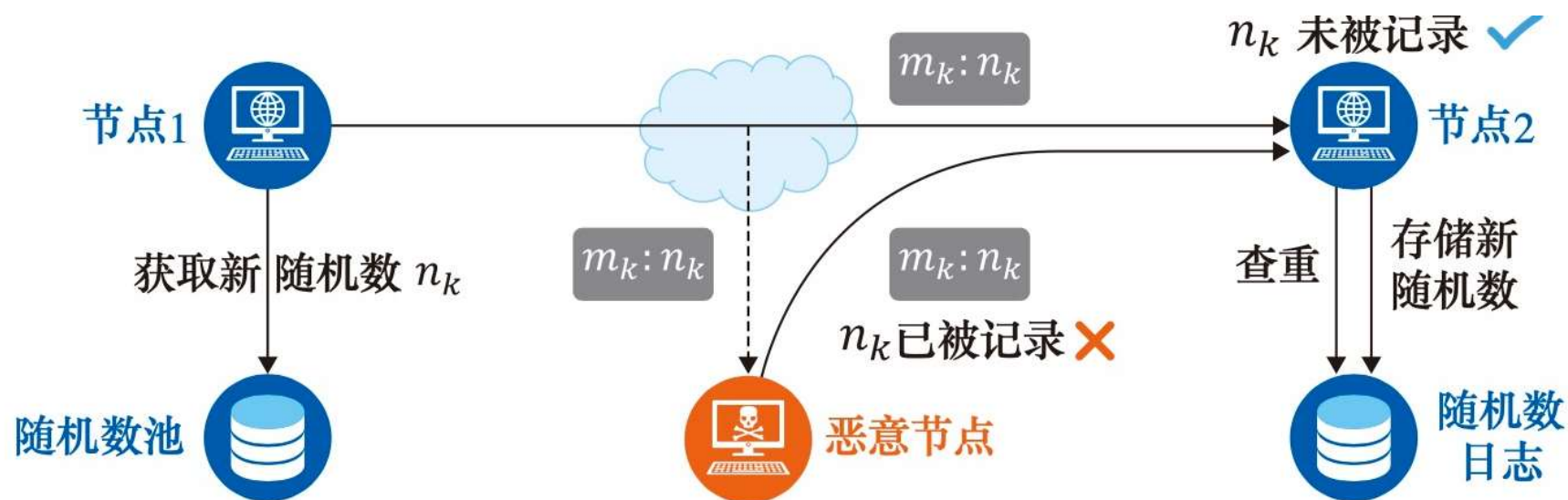
優點：容易實現

缺點：要求交互雙方進行精確的時鐘同步；存在一個可以被攻擊的時間窗口



基于隨機數防禦重放攻擊

- 節點1維護隨機數池，和節點2交互的每條消息都附帶新隨機數
- 節點2收到消息後，判斷該消息所附帶的隨機數是否已在日志R中
若否，則接受該消息，并將新隨機數存儲在本地日志；若是，則丟棄該消息



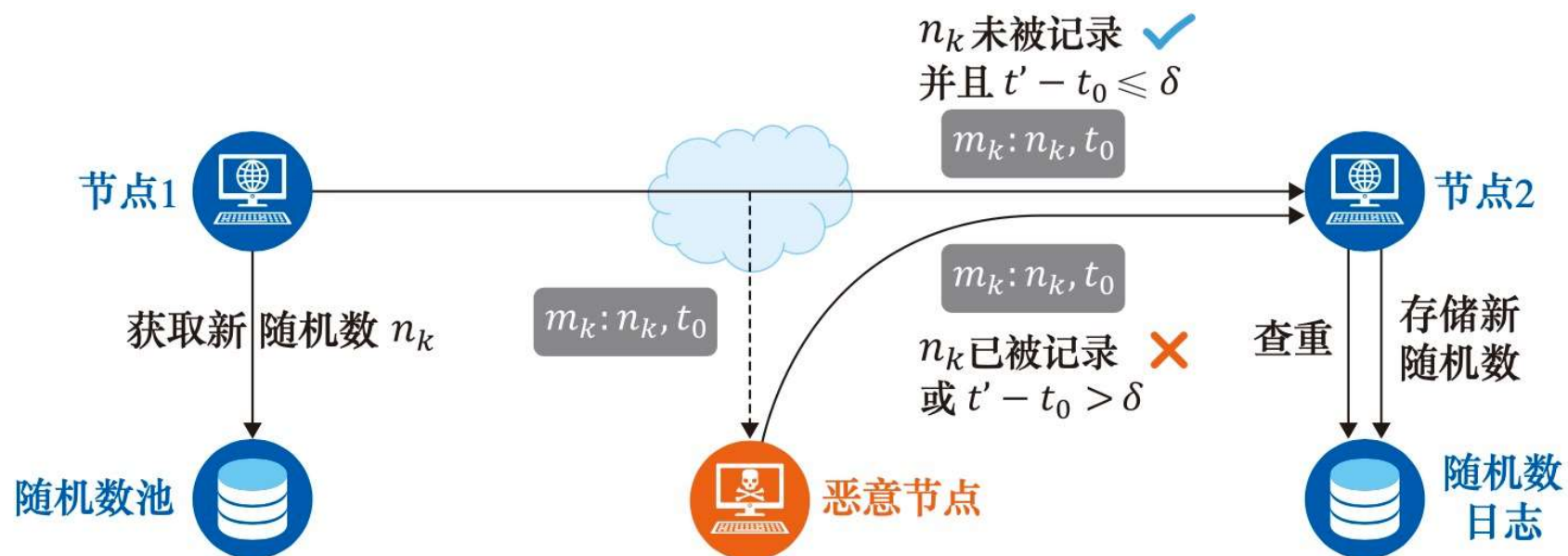
優點：無需時鐘同步；不存在可被攻擊的時間窗口

缺點：複用之前的數時，可能遭受重放攻擊；需要記錄大量已用過的隨機數



時間戳和隨機數相結合

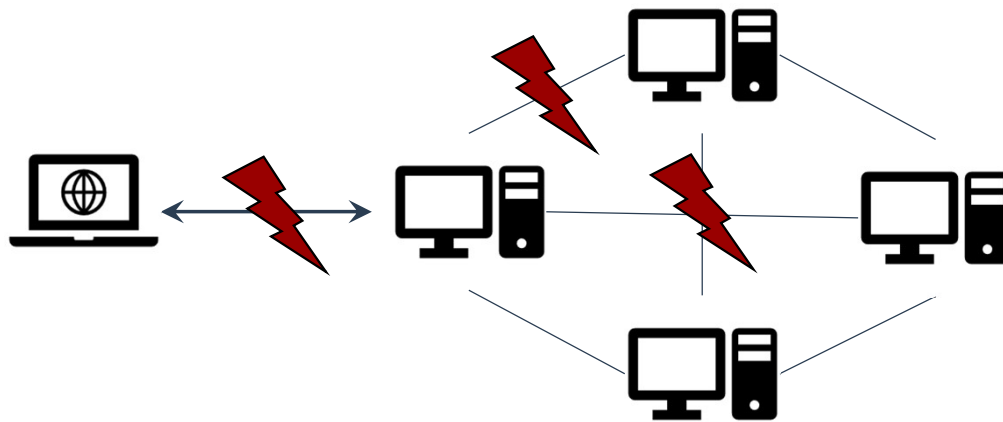
- 節點1和節點2時鐘同步，無需非常精確
- 節點1維護隨機數池，隨機數的量確保可以在短時間內不重複
- 節點2只存儲短時間內的隨機數
- 對消息的判斷需同時判斷時間戳和隨機數





當網絡鏈路出現故障

- 鏈路故障類型
 - 鏈路崩潰:** 路由配置、BGP劫持等問題, 使主機無法和部分其它域的主機通信
 - 性能問題:** 網絡攻擊導致主機和部分其它域主機通信的延遲顯著上升



- 鏈路故障的特徵
 - 無法通過網絡層路由解決
 - 通常只會影響節點與部分節點間的通信信道, 不會造成網絡分區



造成網絡故障的主要原因

Internet的不穩定性，分布式系統所依賴的底層網絡隨時都可能出現物理鏈路故障、路由故障等因素

Paxson 95-97	• 3.3% 的路由存在嚴重問題
Labovitz 97-00	• 10% 的路由有效時間不超過 95% • 65% 的路由有效時間不超過 99.9% • 路由故障的檢測與恢復至少耗費3分鐘，平均花費15分鐘 • 40% 的鏈路崩潰需要至少30分鐘才能修復
Chandra 01	• 5% 的鏈路錯誤持續2.75小時以上



解决思路

构建一个弹性覆盖网络，依靠其它主机实现消息的转发



覆蓋網絡

Overlay Network

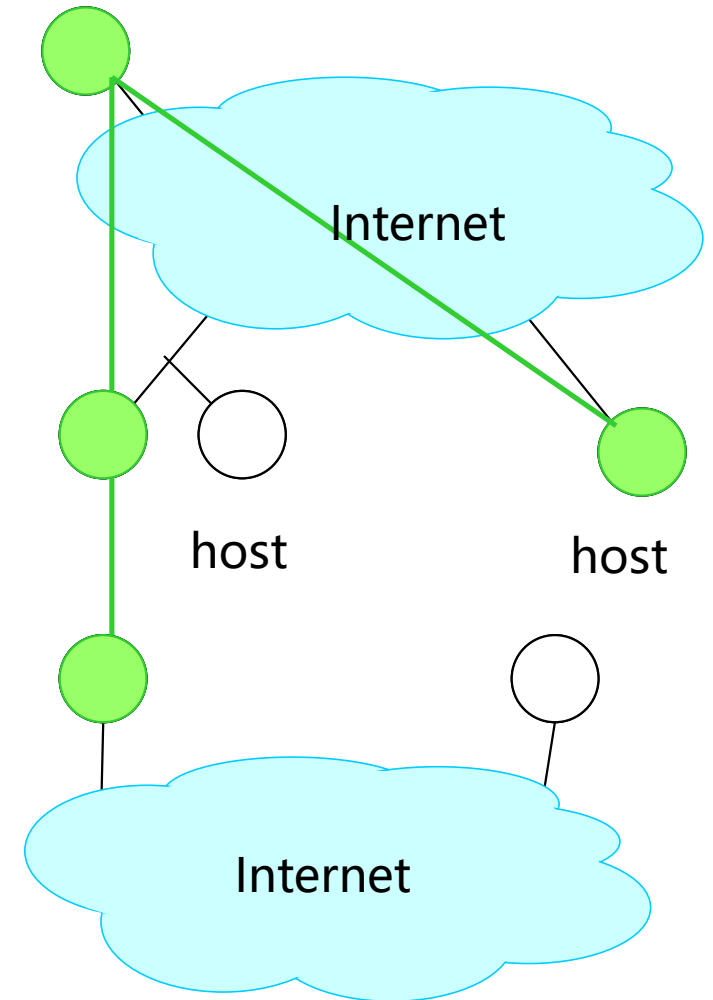
現有Internet網絡上構建一個完全位于應用層的網絡系統



拓撲發現，路由等功能完全由應用層完成，不依賴網絡層



基于Internet網絡的大規模分布式應用

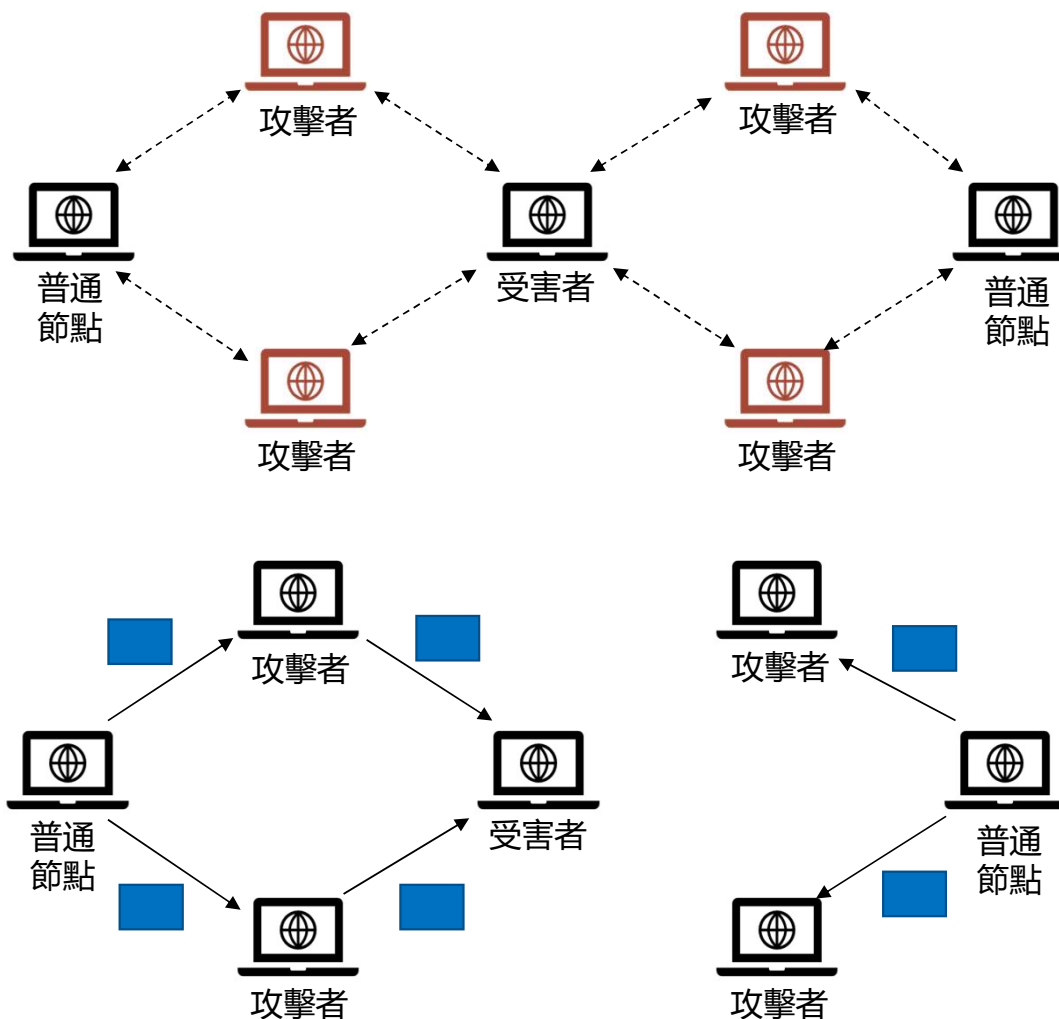




日蝕攻擊

攻擊目標：針對網絡節點發現以及鄰居節點維護兩個過程進行攻擊，使得受害節點的所有鄰居節點都是由攻擊者所控制的惡意節點

危害：攻擊者可以選擇性的轉發對攻擊者有利的消息給受害者，從而配合其它網絡攻擊；攻擊者可以完全隔離受害者與網絡中其它節點的信息交互



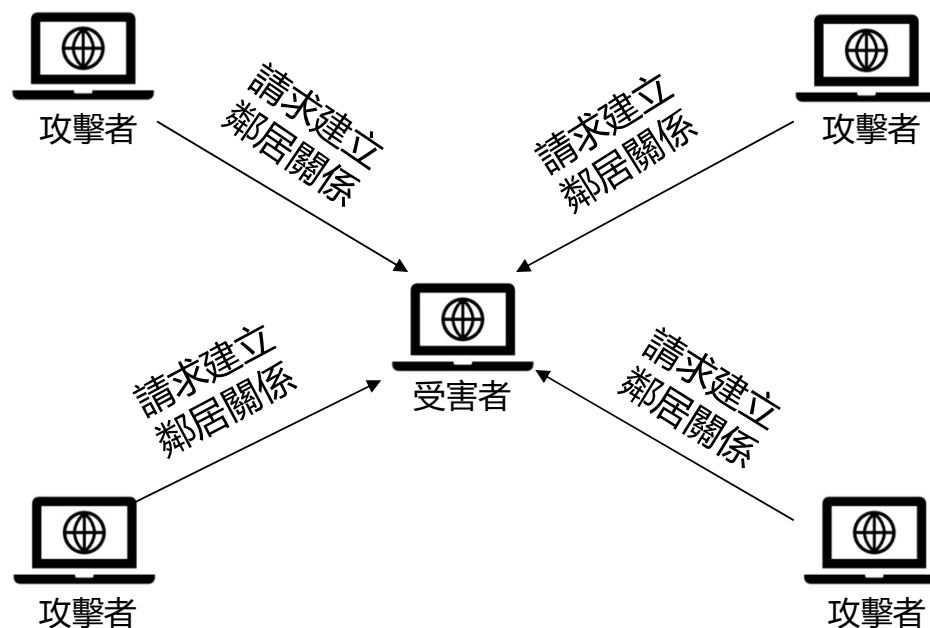


擠占入向連接

主要思路

- 攻擊者控制大量節點主動向受害者請求建立鄰居關係
- 若受害者已經和其它節點建立鄰居關係，則可配合DDoS等其它網絡攻擊觸發受害節點的重啓，然後迅速擠占其入向連接

解決思路：
周期性更換鄰居節點





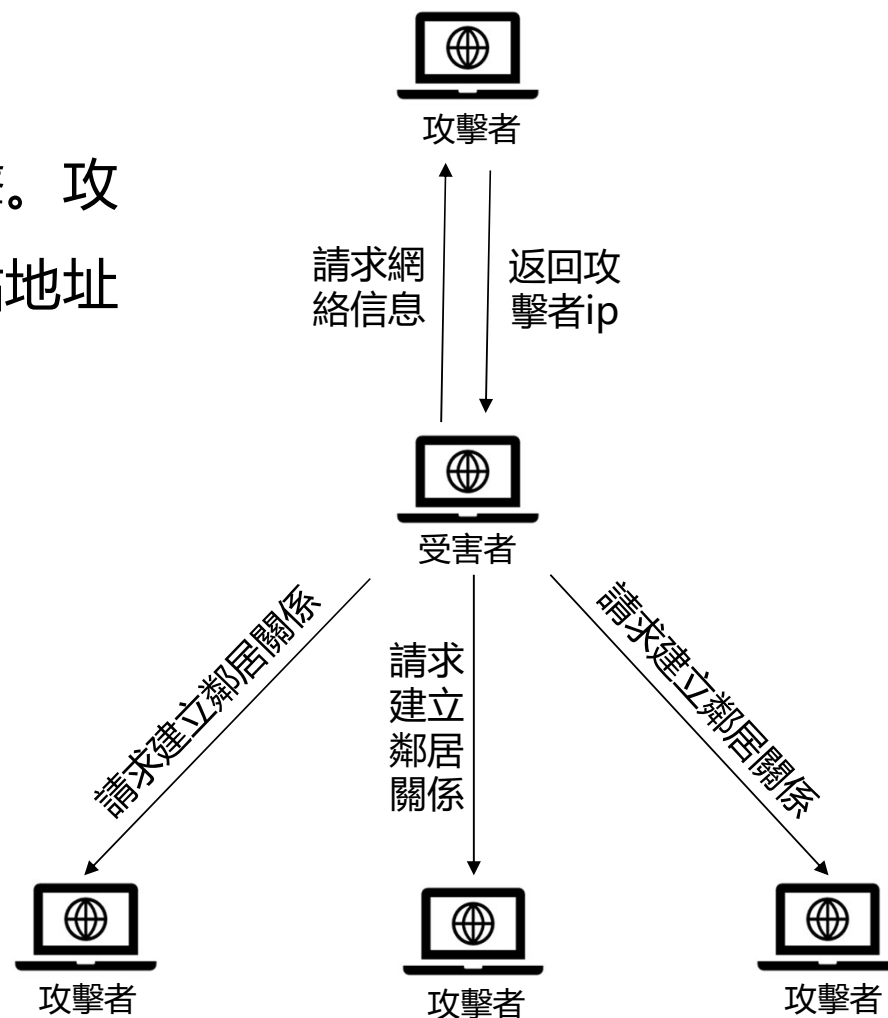
擠占出向連接

主要思路：

針對受害節點發現網絡節點的過程進行攻擊。攻擊者向受害者發送大量由自己所控制的節點地址信息，等待受害者主動請求建立連接

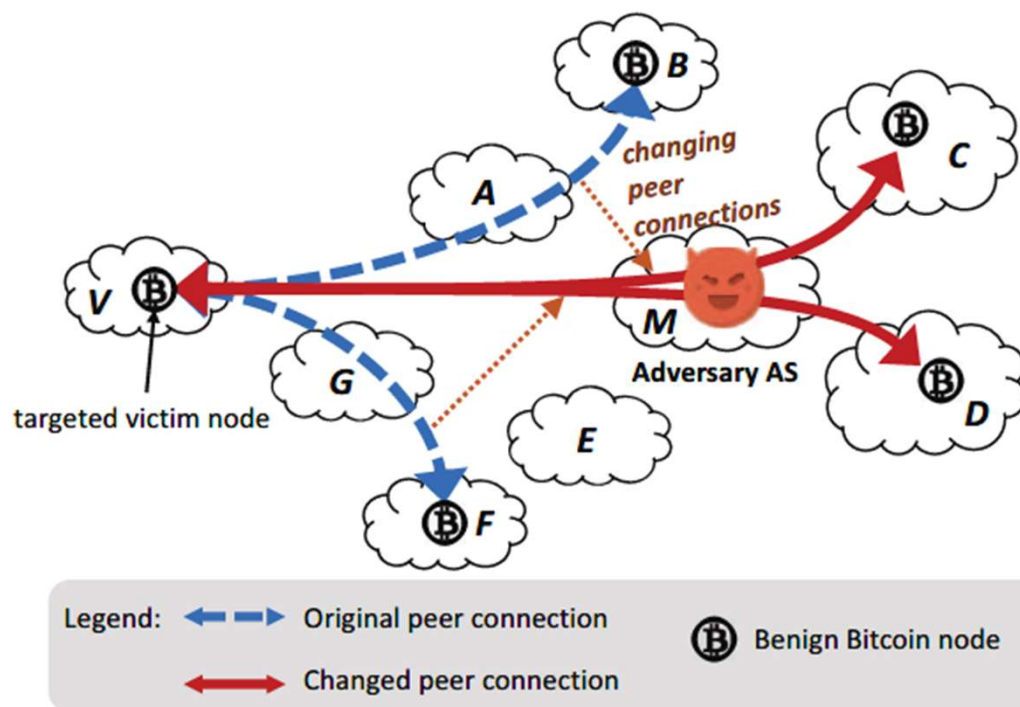
解決思路：

- 增加網絡信息表項的大小
- 增加出向連接個數
- 選擇少量不易受攻擊的可信權威節點作為鄰居節點





配合路由劫持攻擊



主要思路：

- 向受害節點本地存儲的節點地址列表注入大量影子IP（與受害節點間的通訊鏈路經過攻擊者所控制的AS）
- 使受害節點與影子IP建立鄰居關係，從而劫持受害者的所有鄰居鏈路

解決思路：

鄰居選擇時將AS拓撲考慮進去

注. 圖片來自來源：

Tran M, Choi I, Moon G J, et al. A stealthier partitioning attack against bitcoin peer-to-peer network[C]//2020 IEEE Symposium on Security and Privacy (S&P). IEEE, 2020: 894-909.



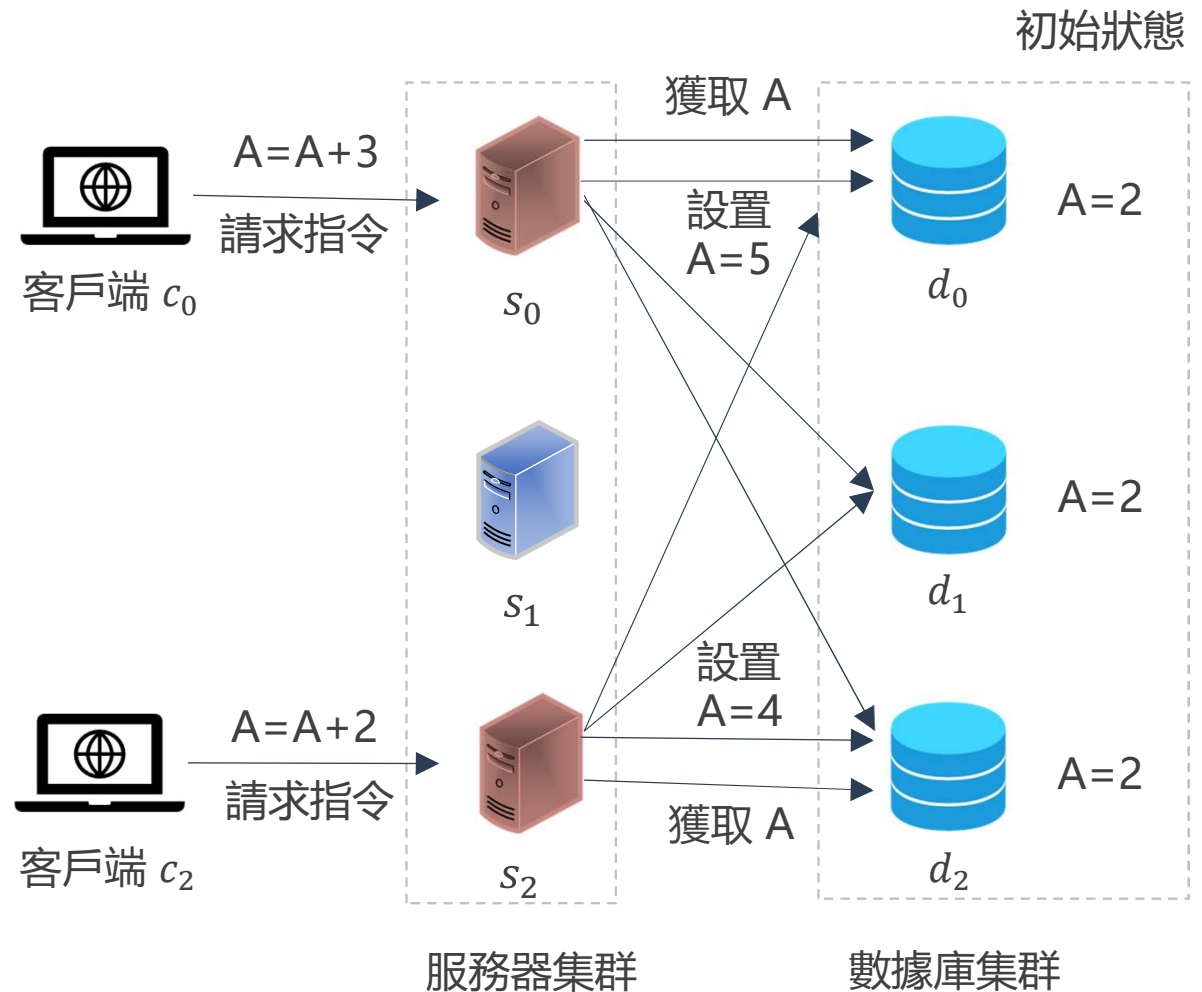
第2節 分布式算法

- ✓ 協作過程中的安全隱患
- ✓ 時鐘同步算法
- ✓ 并發問題
- ✓ 故障容錯的一致性算法



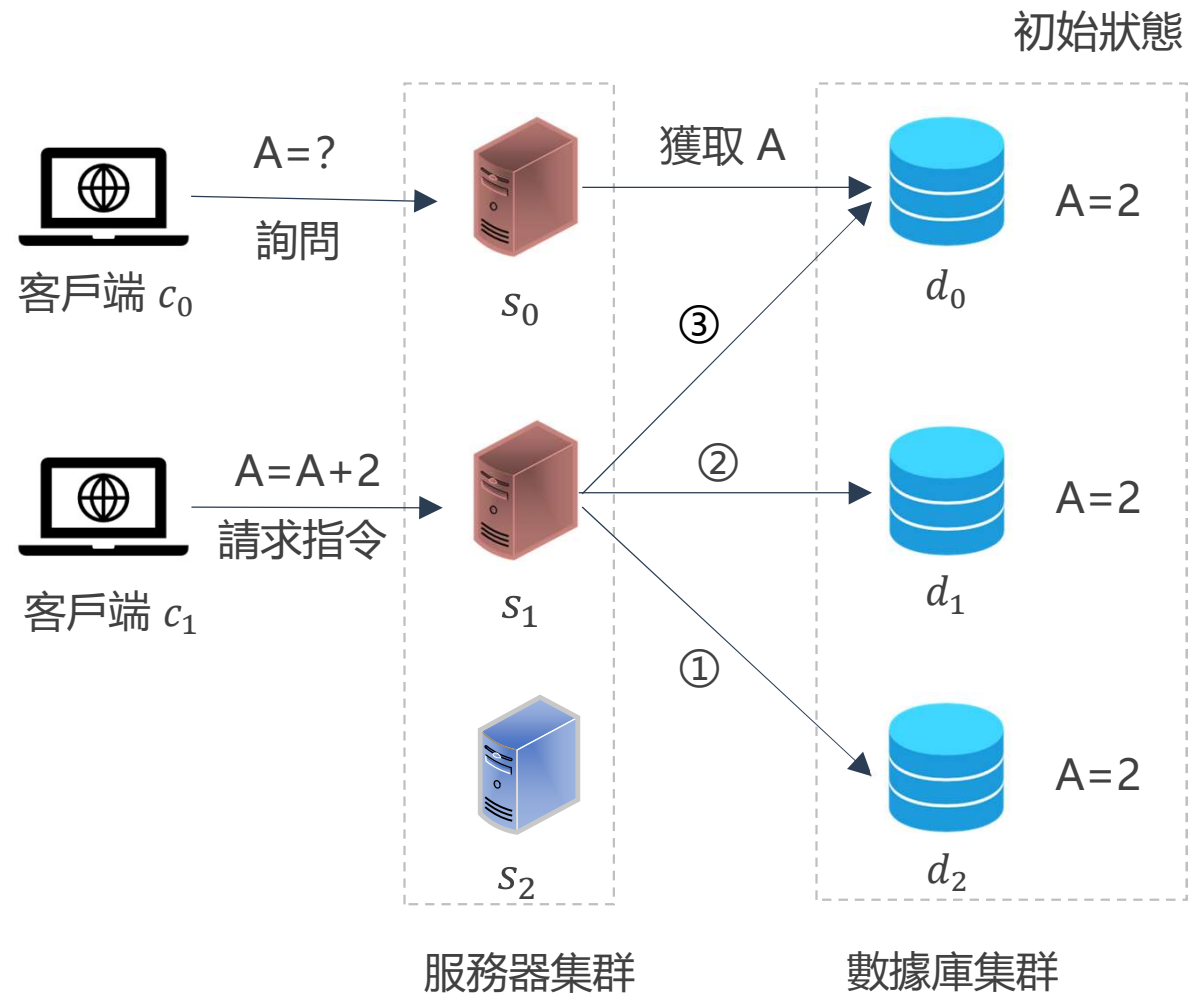
并發問題：破壞一致性

- 假設：狀態 A 在三個數據庫冗餘存儲，不存在并發控制措施
- 執行步驟
 - c_0 向 s_0 請求將 A 的值加3
 - c_2 向 s_2 請求將 A 的值加2
 - s_0 和 s_2 分別向 d_0 和 d_2 讀取 A 的值并完成計算
 - s_0 和 s_2 依次向三個數據庫更新 A 的值
- 由于指令的并發執行，可能導致三個數據庫 A 的值不一致



并發問題：指令之間相互影響

- 假設：狀態 A 在三個數據庫冗餘存儲，不存在并發控制措施
- 執行步驟
 - c_0 向 s_0 請求 A 的值
 - c_1 向 s_1 請求將 A 的值加2
 - s_1 依次將三個數據庫中 A 的值修改為4
 - s_0 向數據庫 d_0 讀取 A 的值
- 由于指令的并發執行， s_0 讀取的值可能為2，也可能為4





事務與分布式事務

事務是原子的一組請求，這組請求不受其它客戶端干擾，并且要麼全部完成，要麼不對服務器造成任何影響

- 事務需要滿足ACID特性
 - 原子性 (Atomicity)：操作必須全部完成，或者不留下任何結果
 - 一致性 (Consistency)：將系統從一個一致狀態轉換到另一個一致狀態
 - 隔離型 (Isolation)：每個事務的執行不受其它事務影響
 - 持久性 (Durability)：一旦事務完成，其效果將被保存到持久存儲中

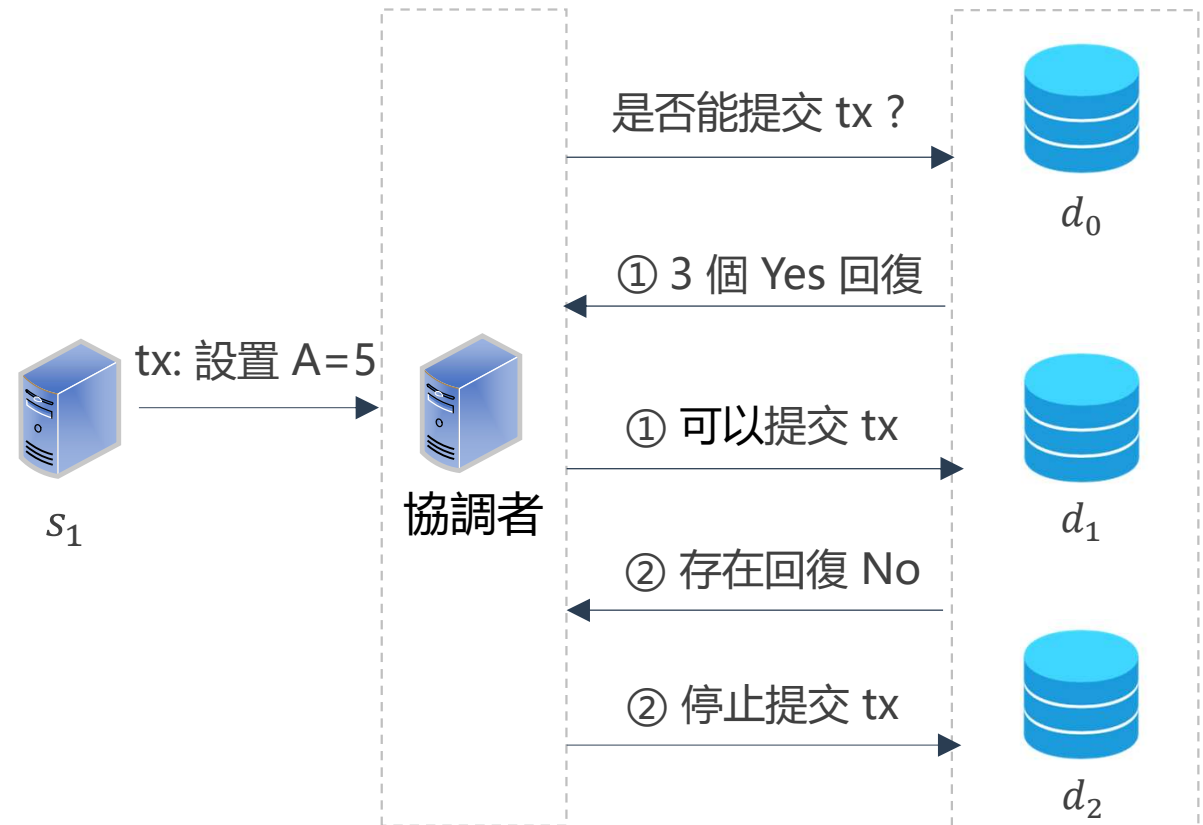
分布式事務是訪問由多個服務器管理的對象的事務



兩階段提交確保一致性

假設：狀態A在三個數據庫冗餘存儲，存在一個協調者引導二階段提交的執行

- 執行步驟
 - s_1 向協調者請求將A設為5
 - 協調者向所有存儲狀態A的數據庫詢問是否可提交事務
 - 數據庫若判斷不衝突，則返回Yes；否則返回No
 - 當協調者收到所有數據庫的Yes，通知它們提交該事務；如果收到No，則通知停止



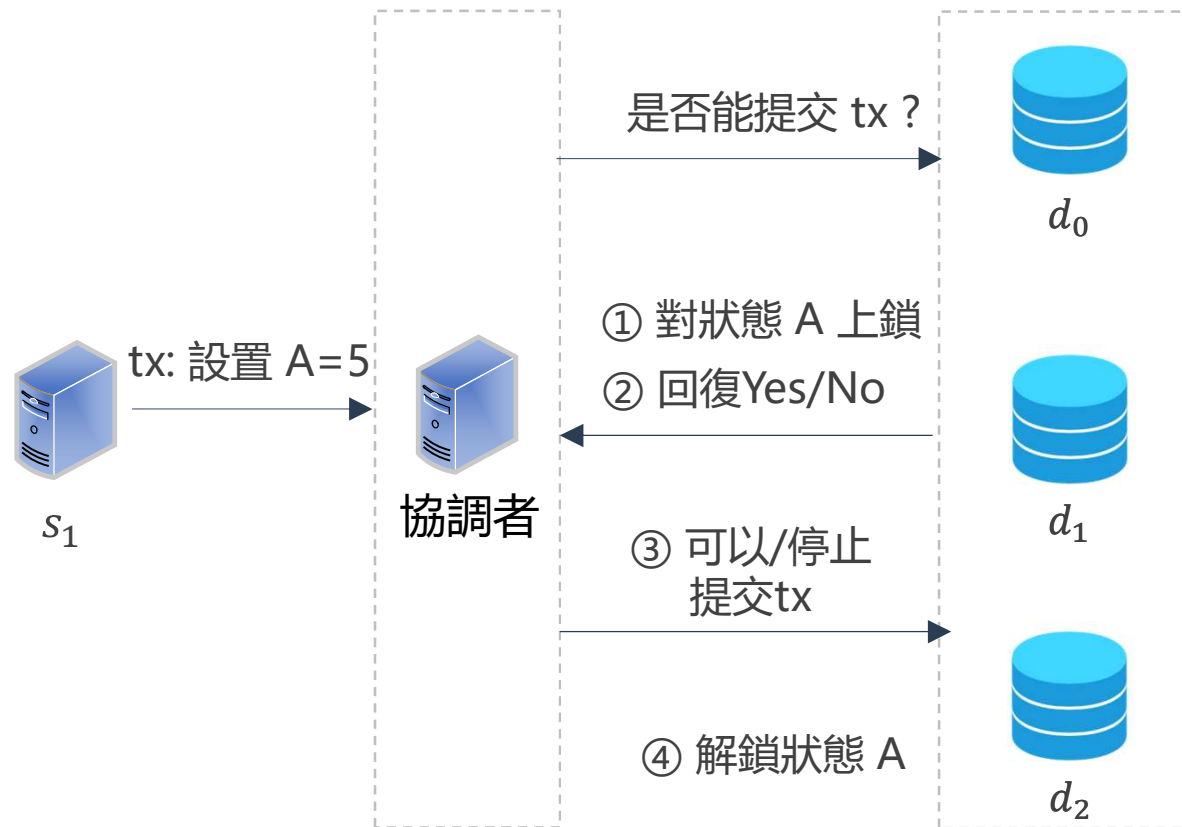


基于鎖機制確保隔離性

假設：狀態A在三個數據庫冗餘存儲，存在一個協調者引導二階段提交的執行

- 執行步驟

- s_1 向協調者請求將A設為5
- 協調者向存儲狀態A的數據庫詢問是否可提交事務
- 數據庫若判斷不衝突，則返回Yes，并對狀態A上鎖
- 數據庫等待協調者的通知，若收到Do commit，完成對狀態A的設置并解鎖狀態A；若收到Abort，則直接解鎖狀態A





討論

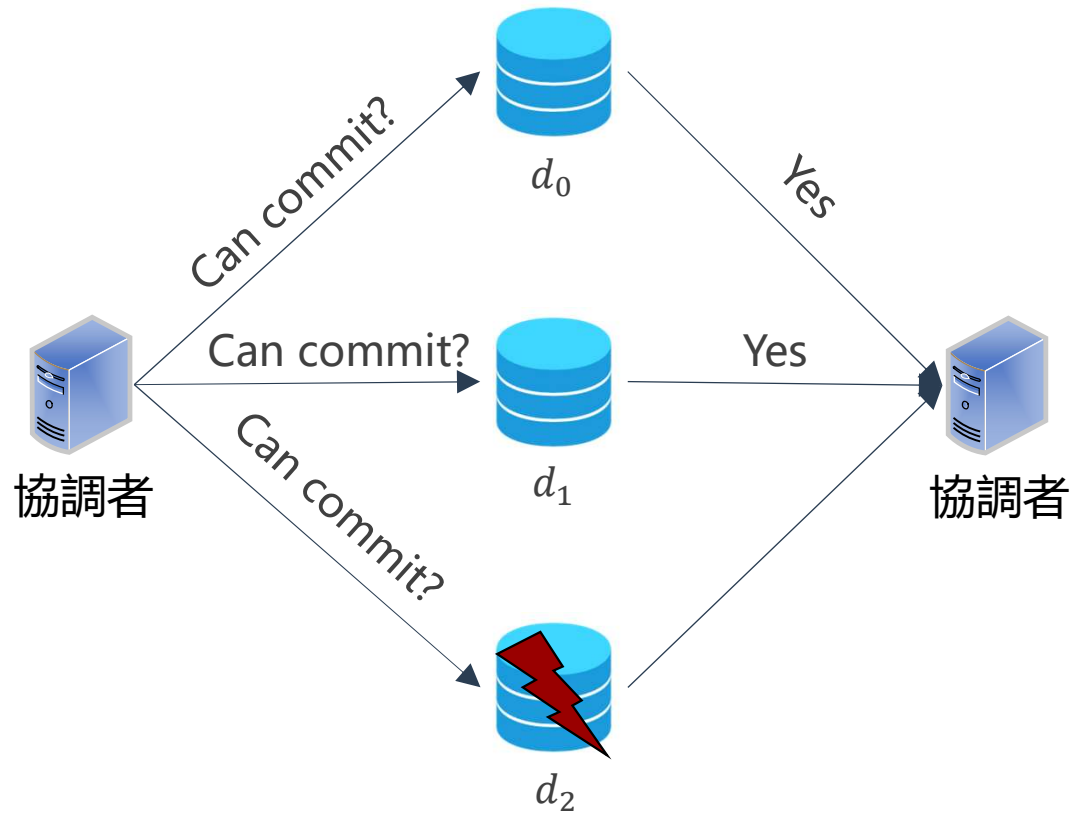
在兩階段提交過程中出現故障怎麼辦？



數據庫節點故障

存在一個協調者和三個存儲相同狀態變量A的數據庫，其中一個數據庫出現故障

- 執行步驟
 - 協調者向三個數據庫詢問是否可以提交一個更改A值的事務
 - 數據庫 d_0 和 d_1 向協調者返回Yes
 - 數據庫 d_2 出現故障，沒有反應
 - 協調者無法收到足夠的Yes，陷入無限等待，或者采用計時機制，超時後通知所有節點停止提交

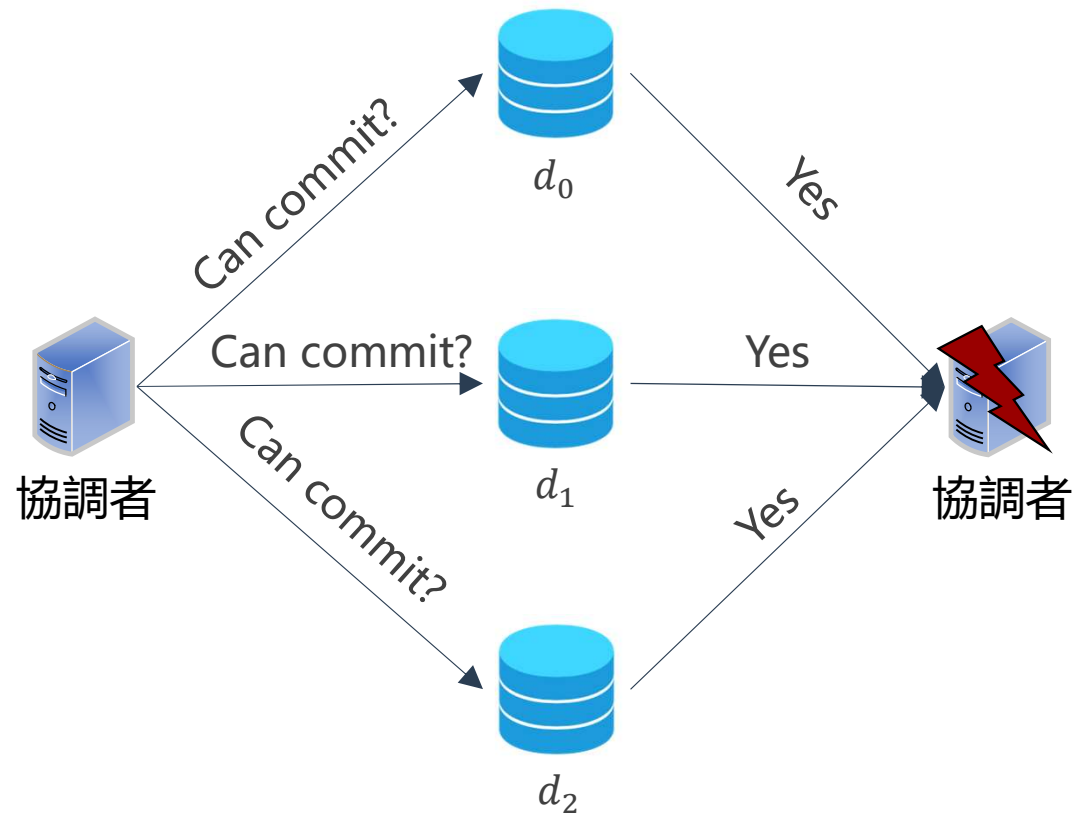




協調者故障

存在一個協調者和三個存儲相同狀態變量A的數據庫

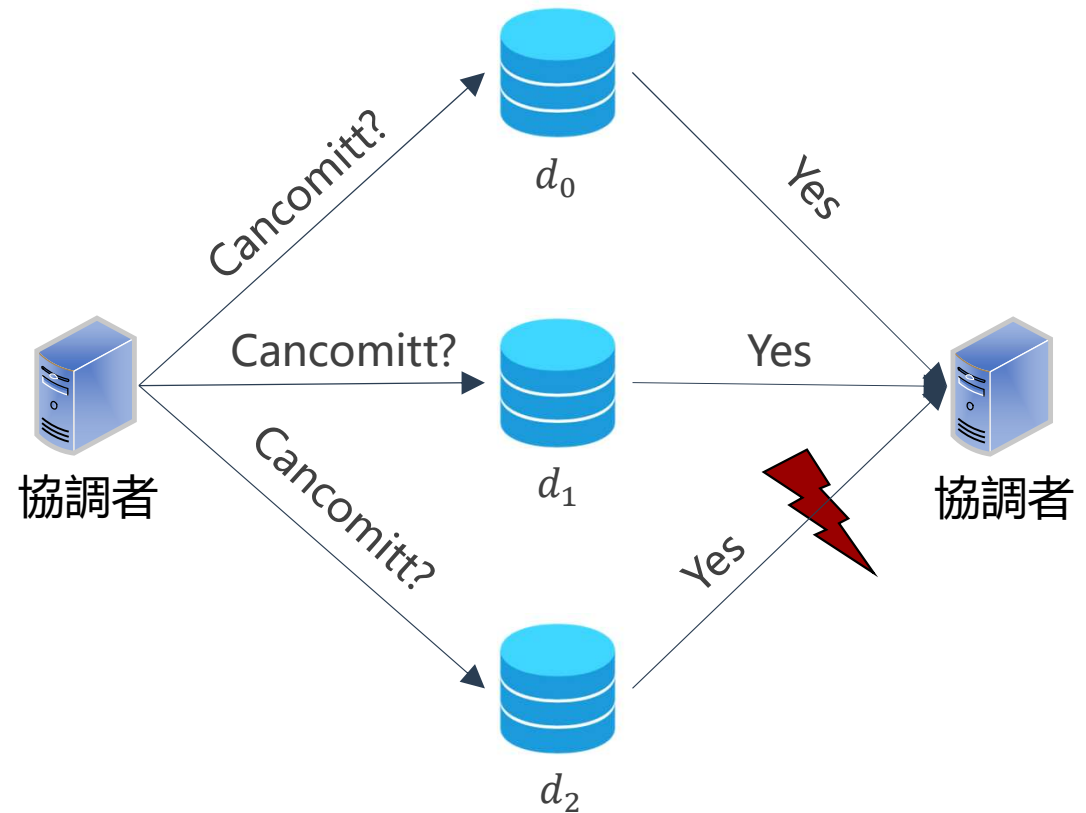
- 執行步驟
 - 協調者向三個數據庫詢問是否可以提交一個更改A值的事務
 - 三個數據庫都向協調者返回Yes
 - 此時協調者出現故障
 - 數據庫一直無法等到協調者的確認提交或停止提交消息，因此一直處於等待狀態，且相關資源一直處於上鎖狀態



網絡擁塞/鏈路故障

存在一個協調者和三個存儲相同狀態變量A的數據庫

- 執行步驟
 - 協調者向三個數據庫詢問是否可以提交一個更改A值的事務
 - 三個數據庫都向協調者返回Yes
 - 數據庫 d_2 與協調者之間的通信鏈路故障，導致Yes消息沒被收到
 - 協調者無法收到足夠的Yes，陷入無限等待，或者采用計時機制，超時後通知所有節點停止提交





無法容錯的根源

在簡單的二階段提交協議中，必須所有相關節點都進行回應後才成功提交，因此協議無法容忍任何故障



印度

我要當常任！

提案

(同意，同意，同意，
同意，反對)

必須五常國全部同意才能通過，只
要保持每次投票的同意票數為4，提
案永遠無法通過

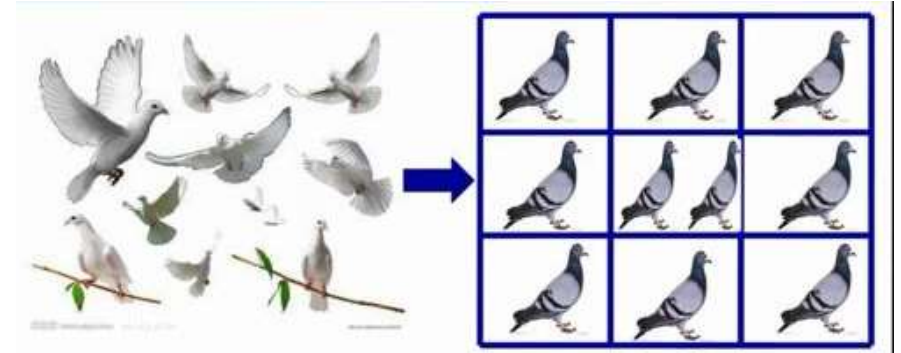


五常國



Quorum設計準則

- **鴿巢原理**：如果 $k+1$ 個鴿子要放在 k 個鴿巢中，那麼至少有一個鴿巢存在兩個鴿子



- 假設 N 個副本，更新操作在 W 個副本中更新成功之後，才認為此次更新操作成功
- 對於讀操作而言，至少需要讀 R 個副本才能讀到此次更新的數據。其中， $W+R>N$ ，即 W 和 R 有重疊，一般， **$W+R=N+1$**
- 假設系統中有5個副本， $W=3$ ， $R=3$ 。初始時數據為 $(V1, V1, V1, V1, V1)$ 當某次更新操作在3個副本上成功後，認為此次更新操作成功，數據變成： $(V2, V2, V2, V1, V1)$ 最多只需要讀3個副本，一定能夠讀到 $V2$ (此次更新成功的數據)



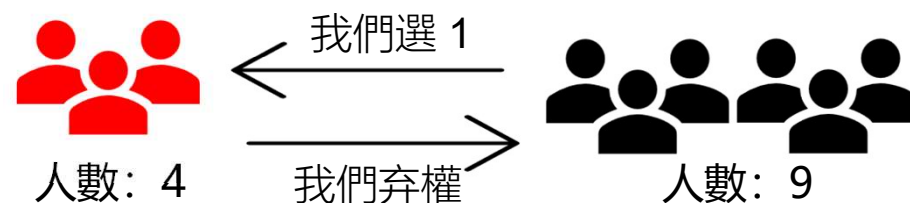
Quorum設計準則

基本規則：分布式系統每階段提案中，誠實節點最多只對一個提案投一次票



q=8 ❌

q=9 ✅



q=10 ❌

q=9 ✅

- 目的1-安全性：Quorum值 q 應該足夠大，避免通過兩個**矛盾提案**

解決方案：針對兩個提案可能產生的最高總投票數 $\leq 2*q - 1$

- 目的2-活性：Quorum值 q 應該足夠小，避免錯誤節點不投票使提案**無法通過**

解決方案： $q \leq$ 最低可能產生的投票數； $q >$ 拜占庭節點個數



故障容錯算法-Paxos

134 • L. Lamport

This submission was recently discovered behind a filing cabinet in the *TOCS* editorial office. Despite its age, the editor-in-chief felt that it was worth publishing. Because the author is currently doing field work in the Greek isles and cannot be reached, I was asked to prepare it for publication.

The author appears to be an archeologist with only a passing interest in computer science. This is unfortunate; even though the obscure ancient Paxon civilization he describes is of little interest to most computer scientists, its legislative system is an excellent model for how to implement a distributed computer system in an asynchronous environment. Indeed, some of the refinements the Paxons made to their protocol appear to be unknown in the systems literature.

The author does give a brief discussion of the Paxon Parliament's relevance to distributed computing in Section 4. Computer scientists will probably want to read that section first. Even before that, they might want to read the explanation of the algorithm for computer scientists by Lamport [1996]. The algorithm is also described more formally by De Prisco et al. [1997]. I have added further comments on the relation between the ancient protocols and more recent work at the end of Section 4.

Keith Marzullo
University of California, San Diego



Leslie B. Lamport

美國著名的計算機科學家，微軟研究院首席研究員，著名的排版系統LaTeX的作者，拜占庭將軍問題和Paxos算法的提出者，2000年獲得了Dijkstra獎，2012年獲得馮·諾依曼獎，**2013年獲得圖靈獎**



故障容錯算法-Paxos

這個世界上只有一種一致性算法，那就是Paxos，其它的算法都是殘次品

——Google Chubby的作者Mike Burrows

- 問題場景：
 - 分布式數據庫系統中，各節點初始狀態一致
 - 部分節點可能出現故障
- 主要目的：
 - 所有正確節點按相同順序執行相同的操作指令
 - 所有正確節點得到一致的本地狀態
- 應用實例：
 - Cassandra, Google Spanner, Google Chubby



Paxos的主要思想

存在有Leader和無Leader兩種模式，基于Quorum機制確保一致性和容錯性

Paxos算法中的3種角色，一個節點可同時擔任多個角色，通常情況下由客戶端擔任提議者，服務端擔任接受者和學習者

- Proposer(提議者), Acceptor(接受者), Learner(學習者)

Paxos的基本流程

- 提議者為交易分配一個序號，從而形成提案，然後將其發送給所有的接受者
- 接受者選擇需要接受的提案，然後對提議者進行回應
- 當提議者收到超過半數接受者的回應後，則通知接受者最終接受該提案
- 接受者接受提案後告知學習者，學習者收到多數接受者信息後學習該提案

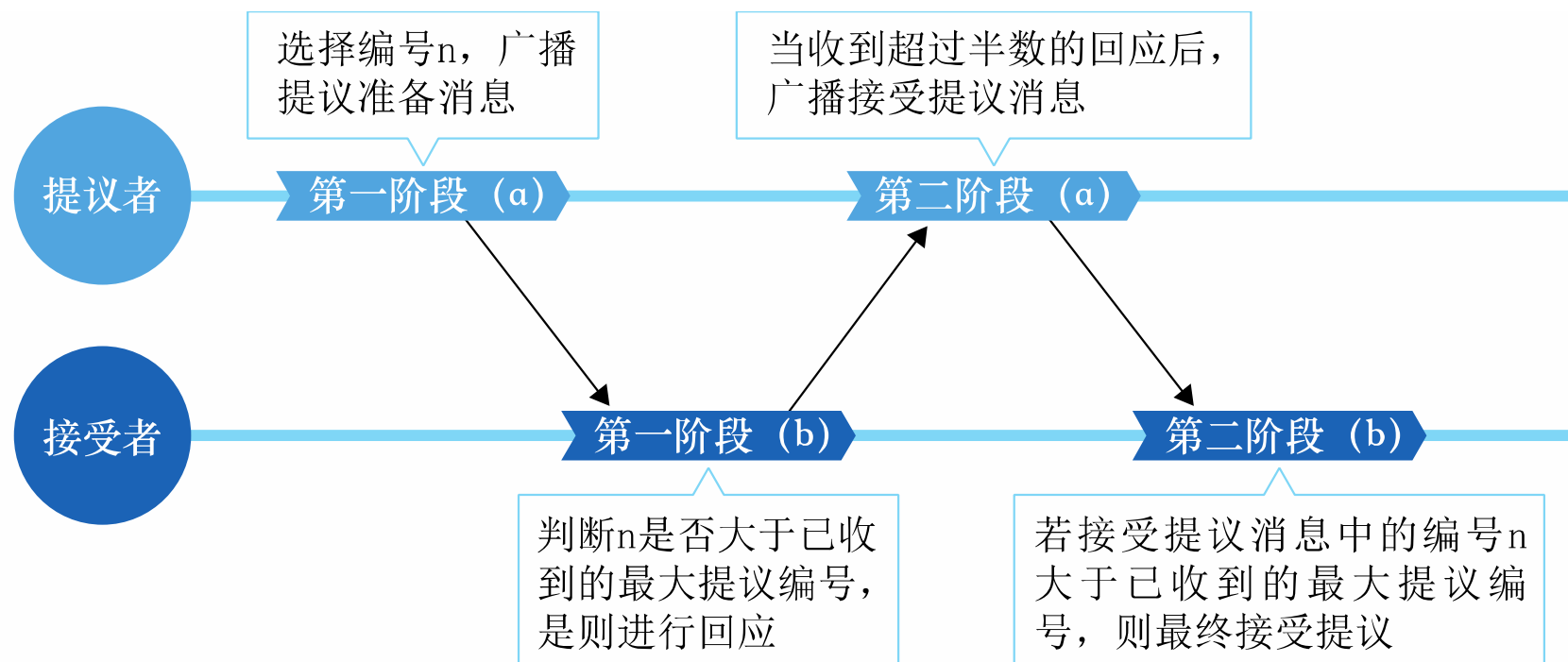
注. 詳情可參考:

Lamport L. Generalized consensus and Paxos[J]. 2005.



無Leader模式下的運流程

一個理想的共識流程



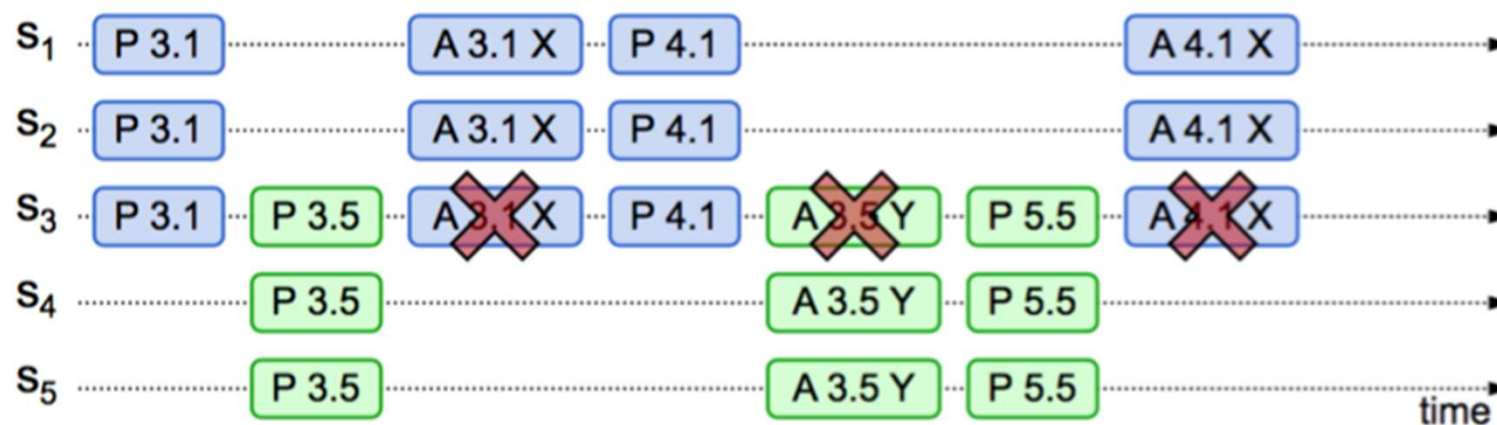
- 實際情況：多個提議者存在，每個提議者單獨提議各自的提案；接受者面臨不同提議者的提案，可能需要不斷接受提議編號更大的提案
- 共識達成條件：提案完成前三次交互之前無更大提議號提案出現



一種永遠無法達成共識的極端場景

每個提議者獨立提議時，不僅效率低下，還可能產生活鎖，即不斷產生衝突

- Competing proposers can livelock:



解決思路：在提議者中選取一個領導者進行提議，其它提議者不參與提議

注. Paxos及更多改進的詳細介紹可參考：

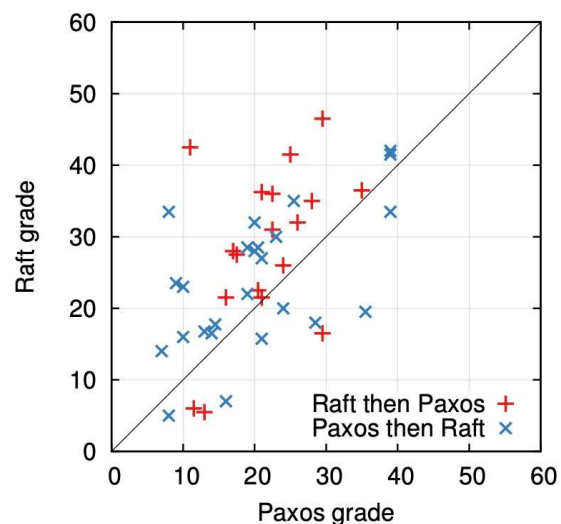
Lamport L. Fast paxos[J]. Distributed Computing, 2006, 19(2): 79-103.



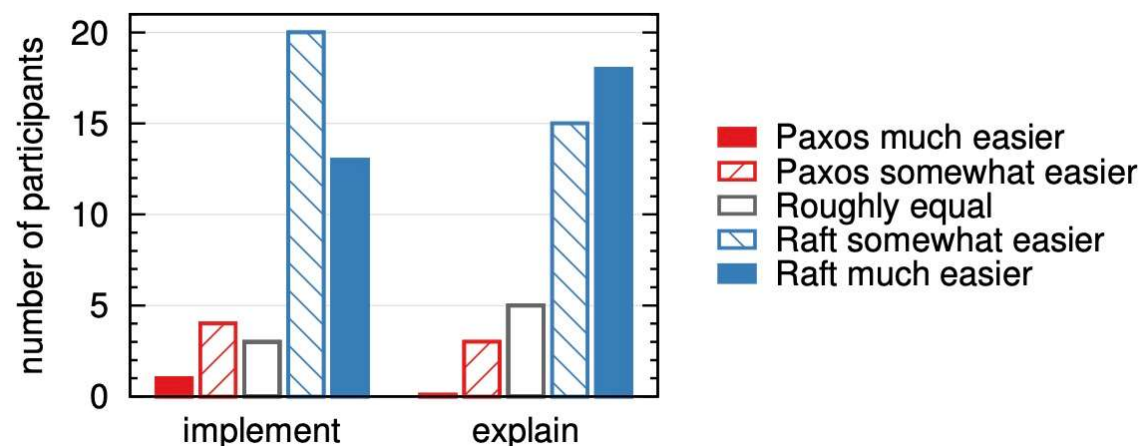
故障容錯算法-Raft

目標：在保證容錯的基礎上，設計一個更容易理解的故障容錯算法。

——Diego Ongaro and John Ousterhout



對參與學生的測試成績



對參與學生的調查問卷

結論：根據測試結果和調查問卷，說明Paxos的學習難度要遠高于Raft



故障容錯算法-Raft

Paxos Made Simple

Leslie Lamport

01 Nov 2001

Abstract

The Paxos algorithm, when presented in plain English, is very simple.

Contents

1	Introduction	1
2	The Consensus Algorithm	1
2.1	The Problem	1
2.2	Choosing a Value	2
2.3	Learning a Chosen Value	6
2.4	Progress	7
2.5	The Implementation	7
3	Implementing a State Machine	8
	References	11



Raft的主要思想

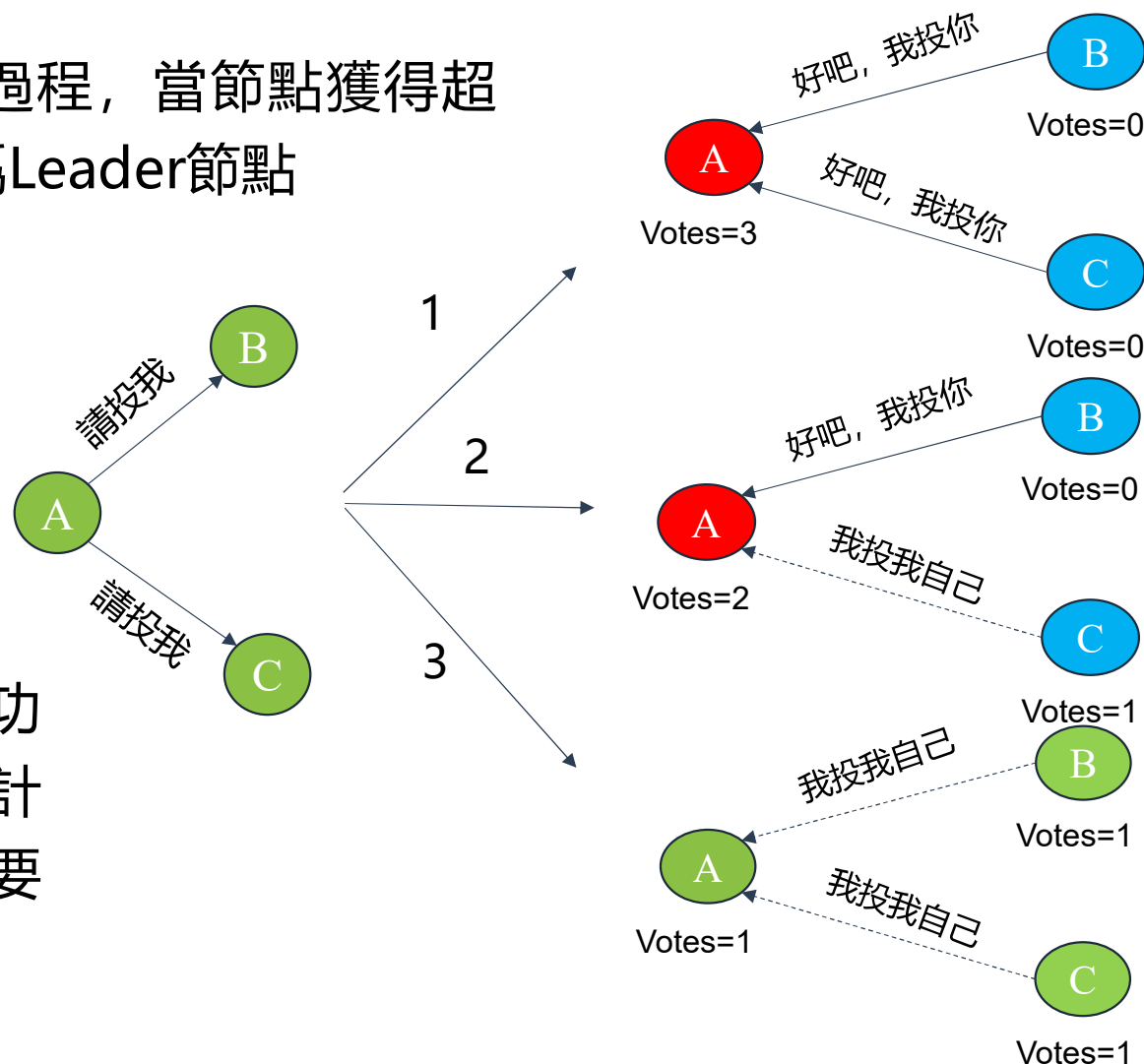
Raft基于Leader完成共識過程，并基于Quorum機制確保一致性和容錯性

- **在Raft中，服務器可以扮演下面角色之一：**
 - Leader: 處理所有客戶端交互，日志複製等，一般一次只有一個Leader
 - Follower: 類似選民，完全被動
 - Candidate: 候選人，可能被選為新的Leader
- **一般狀態下Raft運行的基本流程：**
 - 所有交易被發往Leader，由Leader為其分配序號發給所有Follower
 - 當收到超過半數Follower的回應時，提交并執行交易，然後回復客戶端
 - Follower提交交易後，在後續的交互中通知Follower提交相同的交易



Raft運行流程

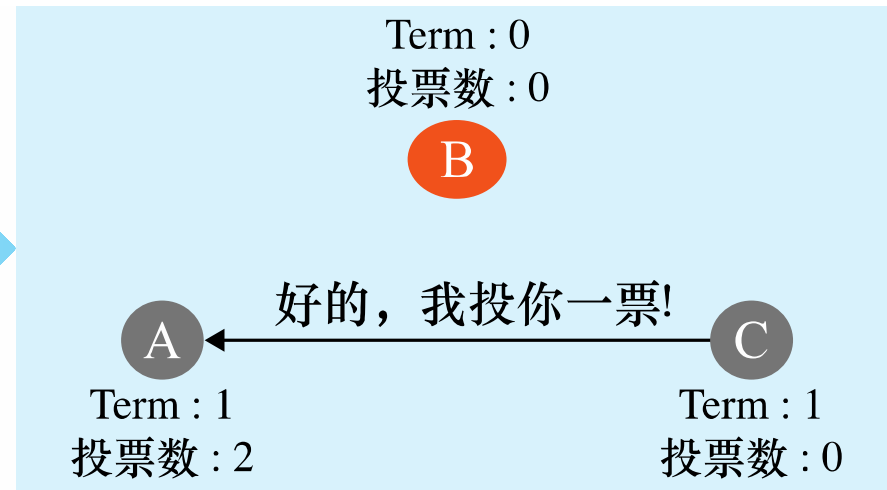
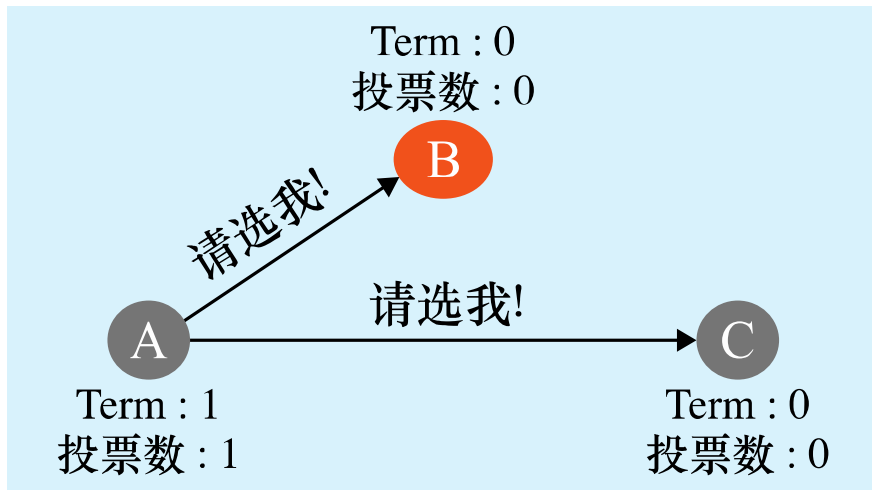
- Raft首先基于投票完成選舉過程，當節點獲得超過半數投票時，則成功競選為Leader節點
- 當競選過程中出現嚴重分票導致無節點成為Leader時，節點觸發超時機制，并進入下一輪Leader選舉
- 爲了確保必然有節點最終成功當選Leader，節點采用隨機計時機制，每個節點超時所需要等待的時間不一樣





Raft運行流程

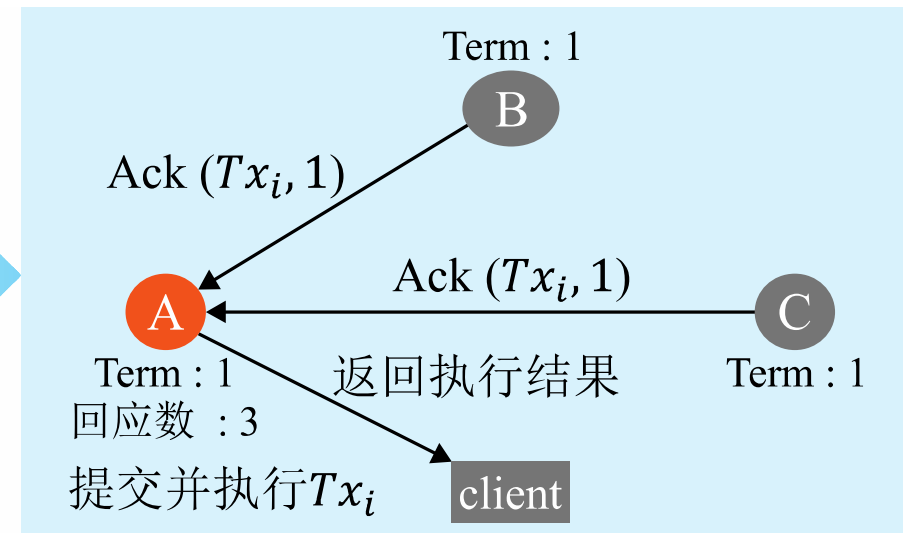
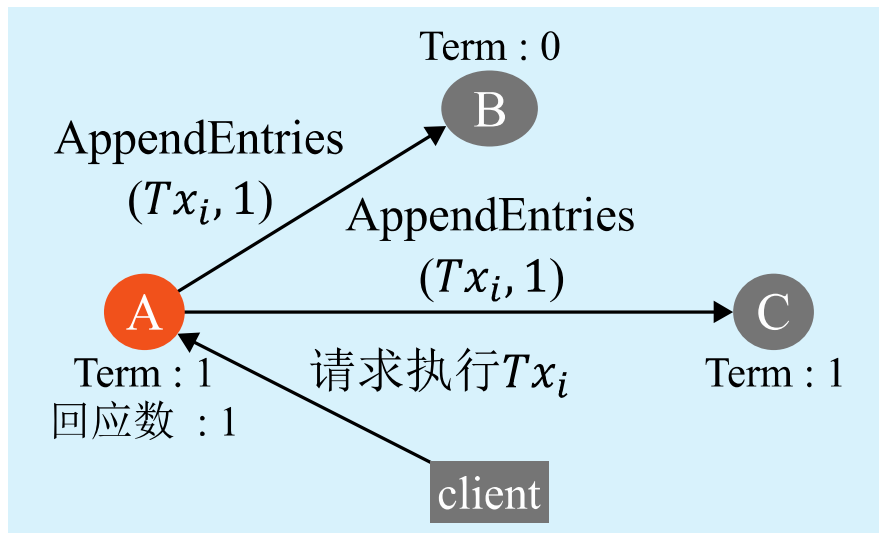
- 第一階段：節點選舉
 - 每個節點包含一個150ms~300ms的計時器
 - 若計時結束後還未從Leader節點收到任何回應，則觸發超時機制
 - 節點將所在的term加1，並成為candidate，向其它節點發送“請選我”的信息，當收到超過半數投票後，則成為leader角色





Raft運行流程

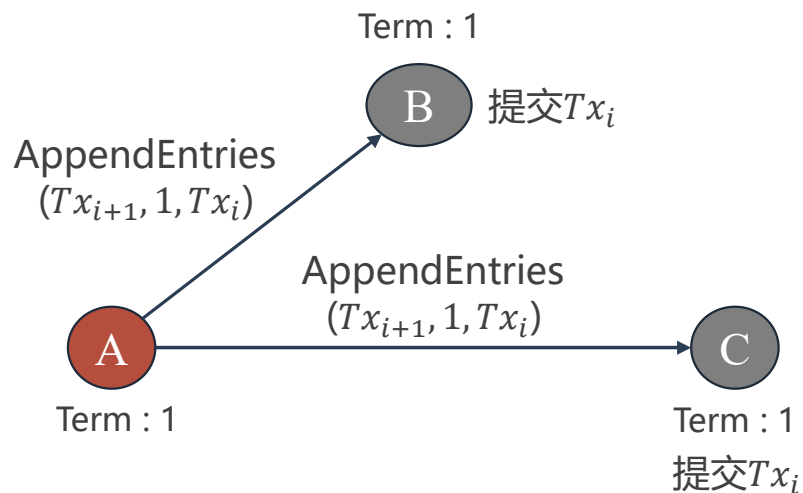
- 第二階段：基于leader的日志複製
 - 客戶端向leader發送交易，等待執行後的結果
 - Leader為交易分配序號并加入本地日志，然後通知其它節點將該交易加入本地日志，當收到大多數節點的Ack後，提交并執行該指令，然後回復客戶端
 - 任意節點，若收到包含更大term的信息，則變為follower，并修改本地term





Raft運行流程

- 第二階段：基于leader的日志複製
 - Leader節點在發送AppendEntries時，會包含最近被提交的交易
 - 其它節點收到AppendEntries後，會跟隨leader提交之前的交易



AppendEntries RPC

Invoked by leader to replicate log entries (§5.3); also used as heartbeat (§5.2).

Arguments:

term	leader's term
leaderId	so follower can redirect clients
prevLogIndex	index of log entry immediately preceding new ones
prevLogTerm	term of prevLogIndex entry
entries[]	log entries to store (empty for heartbeat; may send more than one for efficiency)
leaderCommit	leader's commitIndex

Results:

term	currentTerm, for leader to update itself
success	true if follower contained entry matching prevLogIndex and prevLogTerm



第3節 信任問題

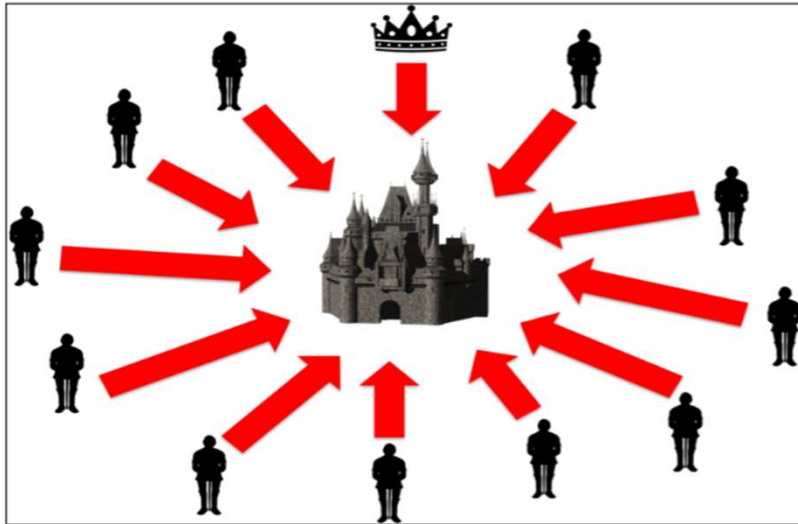


- ✓ 訪問控制
- ✓ 信任評價模型
- ✓ 拜占庭容錯

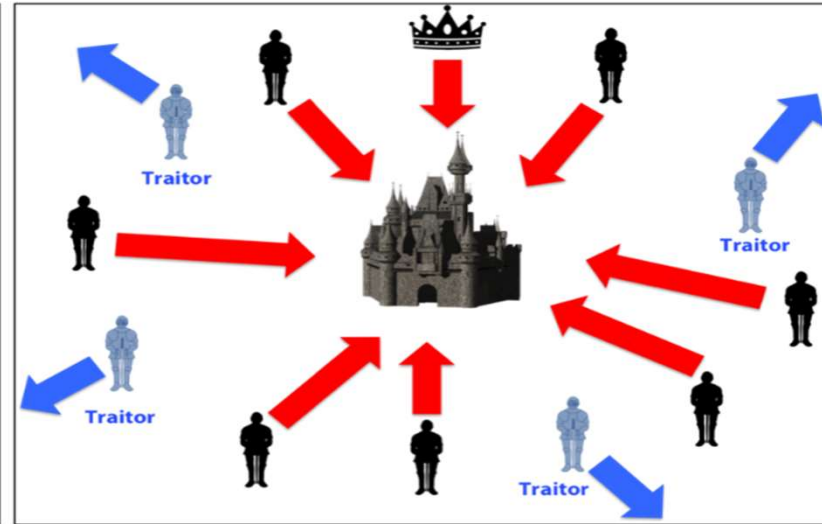


拜占庭將軍問題

- **背景：**拜占庭羅馬帝國疆域遼闊，許多將軍守衛疆土；戰時，部隊相隔遠，只能靠信差傳遞消息；至少超過半數的將軍對是否共打敵人達成一致，否則失敗；內部可能存在叛徒
- **問題：**存在叛徒時，忠誠的將軍們如何達成共識



Coordinated Attack Leading to Victory



Uncoordinated Attack Leading to Defeat



拜占庭將軍問題

- **概念：**將軍 等價于 計算機系統的部件
- **問題抽象：**
 - 每個部隊由一個將軍領導
 - 一共有 n 個將軍，其中一些是叛徒
 - 所有軍隊將敵方包圍，觀察敵軍
 - 將軍間靠通信兵交流



- **目標：**存在所有忠誠的將軍采取**同樣**的行動；**少量**叛徒無法使誠實的將軍們接受一個**壞**的計劃
- **注意：**不必識別出叛徒



拜占庭將軍問題

The Byzantine Generals Problem

Leslie Lamport (SRI International),
Robert Shostak (SRI International),
Marshall Pease (SRI International)

Reliable computer systems must handle malfunctioning components that give conflicting information to different parts of the system. This situation can be expressed abstractly in terms of a group of generals of the Byzantine army camped with their troops around an enemy city. Communicating only by messenger, the generals must agree upon a common battle plan. However, one or more of them may be traitors who will try to confuse the others. The problem is to find an algorithm to ensure that the loyal generals will reach agreement. It is shown that, using only oral messages, this problem is solvable if and only if more than two-thirds of the generals are loyal; so a single traitor can confound two loyal generals. With unforgeable written messages, the problem is solvable for any number of

This research was supported in part by the National Aeronautics and Space Administration under contract NAS1-15428 Mod. 3, the Ballistic Missile Defense Systems Command under contract DASG60-78-C-0046, and the Army Research Office under contract DAAG29-79-C-0102.

Authors' address: Computer Science Laboratory, SRI International, 333 Ravenswood Avenue, Menlo Park, CA 94025.

Permission to copy without fee all or part of this material is granted provided that the copies are not made or distributed for direct commercial advantage, the ACM copyright notice and the title of the publication and its date appear, and notice is given that copying is by permission of the Association for Computing Machinery. To copy otherwise, or to republish, requires a fee and/or specific permission.

© 1982 ACM 0164-0925/82/0700-0382 \$00.75

Paper originally published in *Transactions on Programming Languages and Systems*, 4(3), July 1982, pp. 382-401.



Leslie B. Lamport

[PDF] [The Byzantine generals problem](#)

L Lamport, R Shostak, M Pease - *Concurrency: the Works of Leslie ...*, 2019 - dl.acm.org

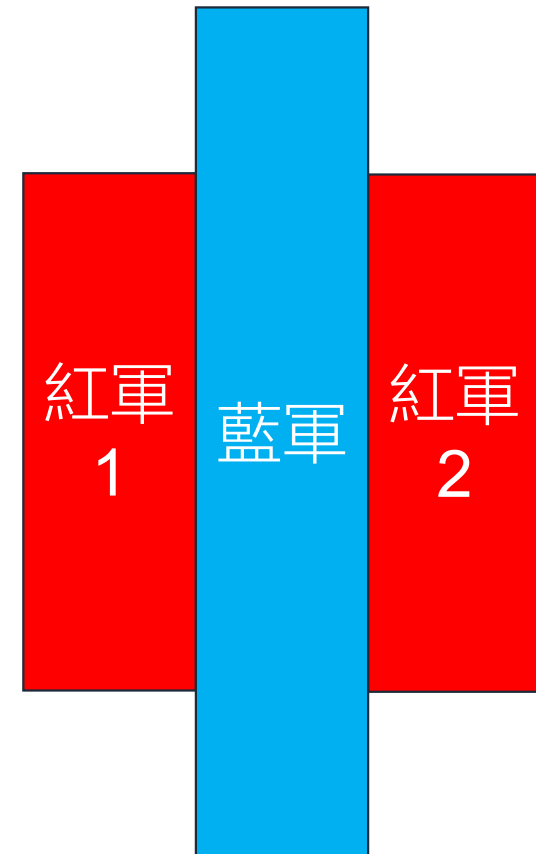
Page 1. The **Byzantine Generals** Problem Leslie Lamport (SRI International), Robert Shostak (SRI International), Marshall Pease (SRI International) ... 382-401. Page 2. 204 The **Byzantine Generals** Problem **generals** and possible traitors ...

☆ 被引用次数: 7814 相关文章 所有 199 个版本



問題定義

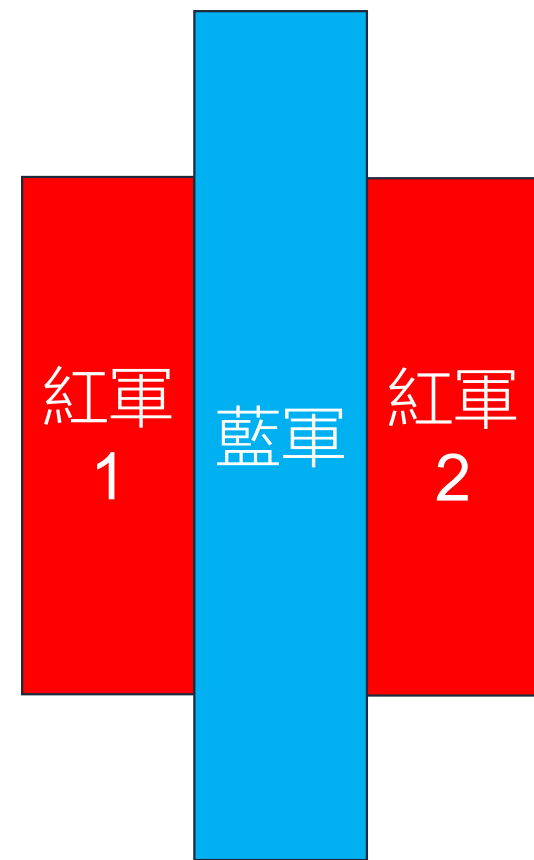
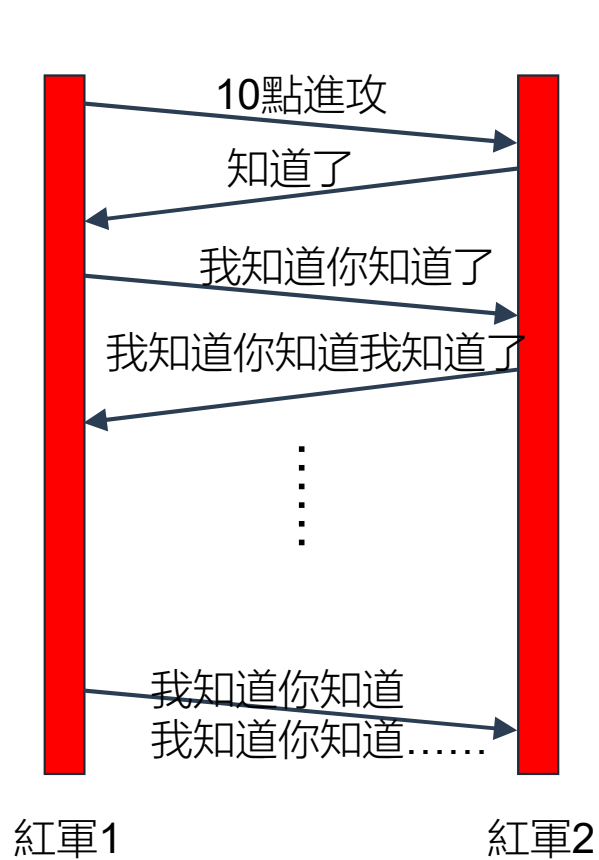
- 拜占庭將軍問題的前提
 - 可靠信道：信息可以準確無誤的傳達給接收方
- 不可靠信道：兩軍問題
 - 紅軍必須在同一時間攻打藍軍才能勝利
 - 紅軍1、2通信必須經過藍軍區域，消息可能被截獲、篡改
 - 如何確定攻打時間？





問題定義

前提條件：不可靠信道：兩軍問題； 結論：兩軍問題不可解

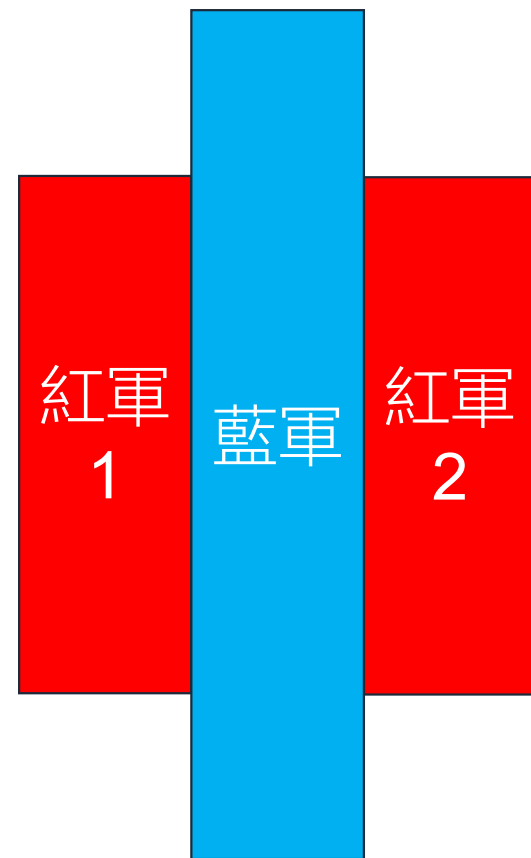
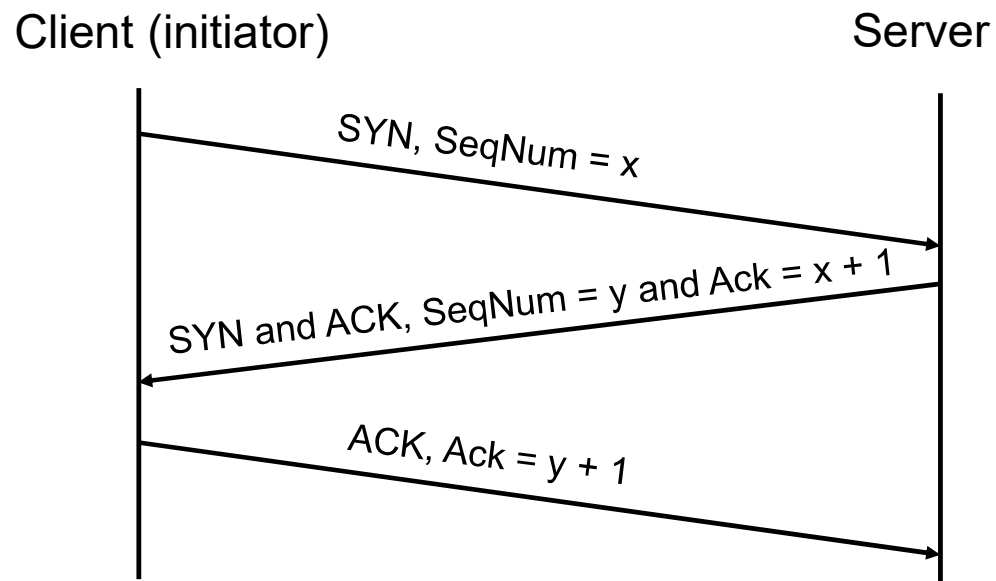




問題定義

兩軍問題實例:TCP通信

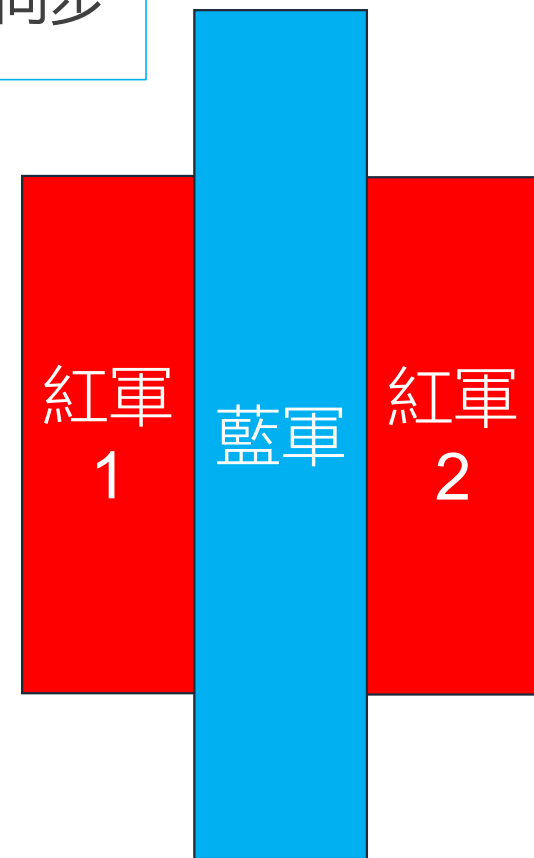
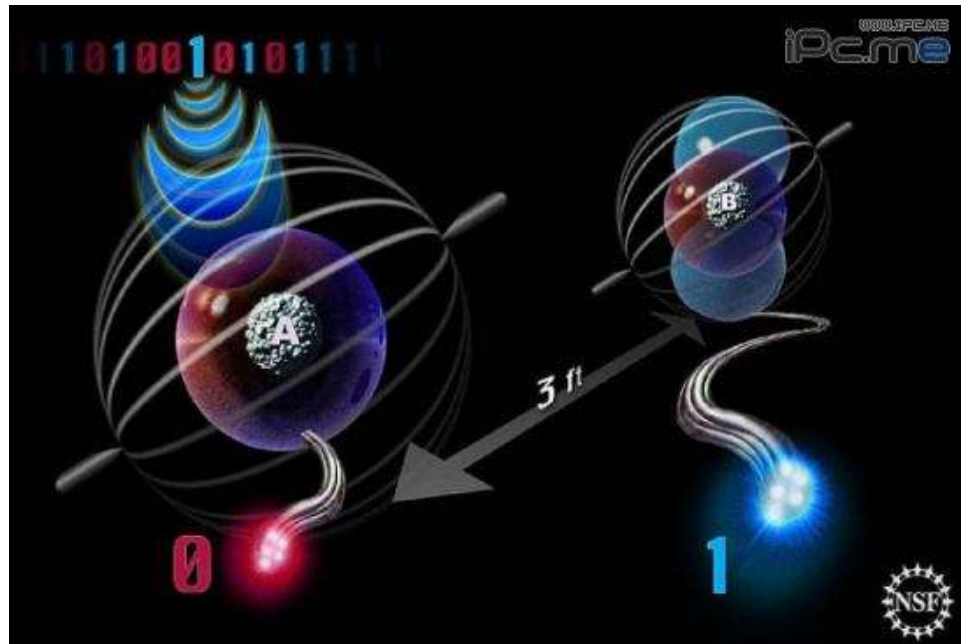
TCP三次握手





問題定義

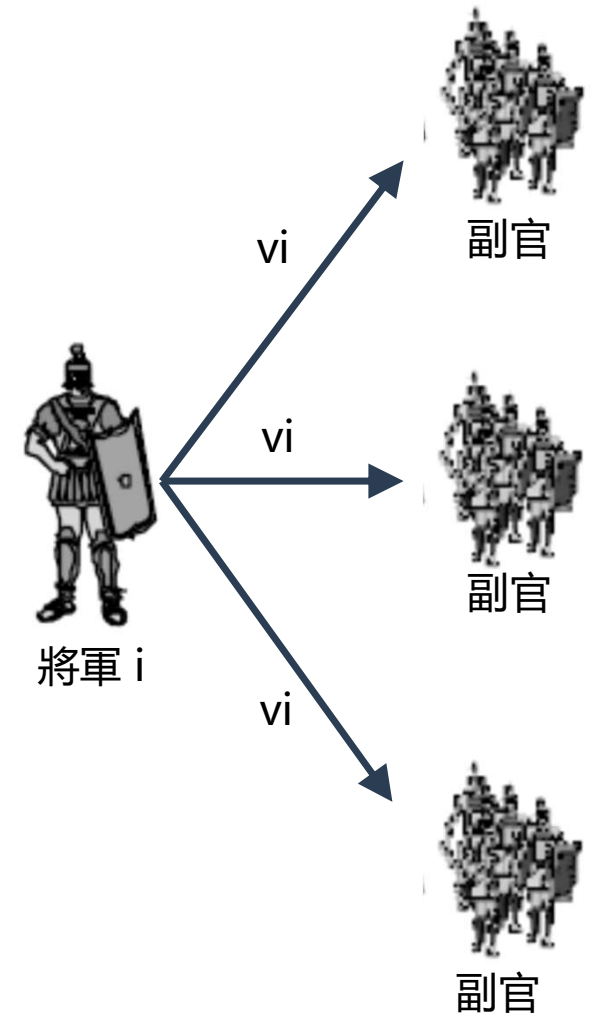
- 兩軍問題終極解決方案：量子通訊
- 處于量子糾纏的兩個粒子，無論相隔多遠都可以同步





問題簡化

- 將問題簡化為：**一個將軍**將消息發送給**其他的副官**
- 拜占庭將軍問題
 - 將軍（發布命令者）必須將他的命令發送給他的 $n-1$ 個副官
- 交互一致性（Interactive Consistency, IC)
 - IC1: 所有忠誠的副官都服從同樣的命令
 - IC2: 如果發布命令的將軍是忠誠的，那麼每個忠誠的副官都服從他的命令
- 注意：如果將軍是忠誠的，那麼 IC2 等價于 IC1





一致性和正確性條件

- 一致性 (等價IC1)
 - 每個將軍必須得到相同的($v_1, v_2, \dots, v_n$)指令向量或指令集合
 - 忠誠的將軍不一定使用 i 將軍發來的信息作為 v_i , i 將軍有可能是叛徒

**一致性條件不夠充分，忠誠將軍的信息有可能被修改，
所以必須引入另一個正確性的條件**

- 正確性 (等價IC2)
 - 若 i 將軍是忠誠的，其他忠誠的將軍必須使用他發出的值作為 v_i



解决方法

主要問題：由于將軍可能是叛徒，會對不同的副官發布不同的指令，因此副官不能完全信任將軍的指令

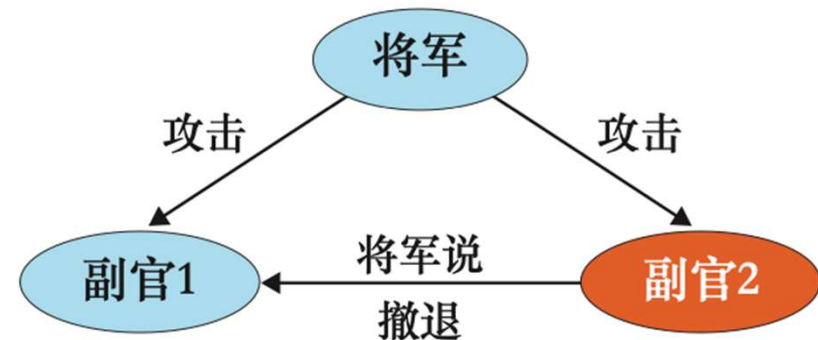
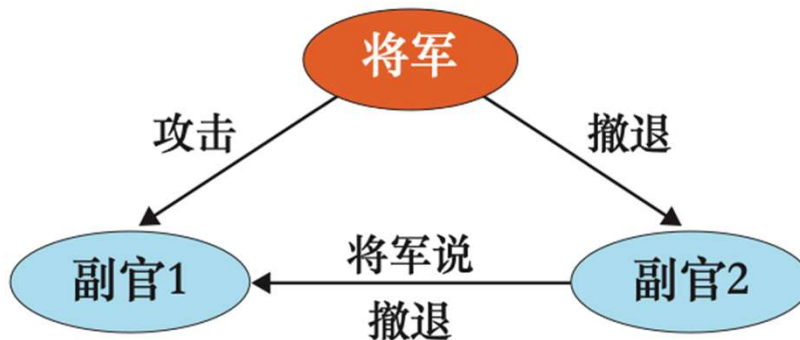
解决思路：副官在收到將軍的指令後，互相交換從將軍處收到的指令值，再基于交互結果選擇最終執行指令

- 兩種方法
 - 口頭消息協議：所有副官在交換從將軍處收到的指令值時基于口頭傳播，因此可以說謊（對應于系統中**無**數字簽名的場景）
 - 簽名消息協議：副官從將軍處收到的指令附帶簽名，因此無法偽造將軍的指令，將軍也無法對已發布的指令抵賴（對應于系統中**有**數字簽名的場景）



口頭消息下三將軍問題無解

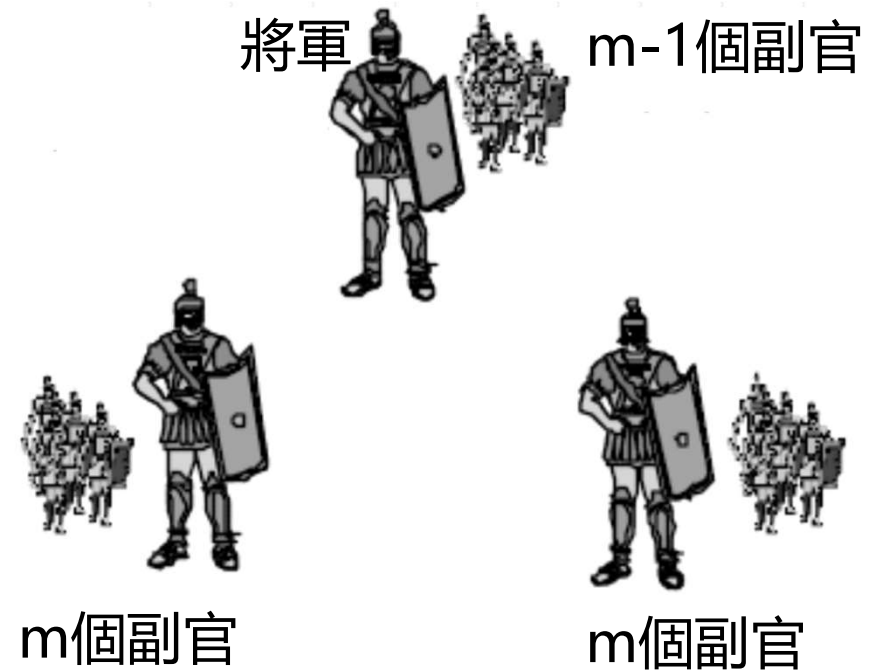
- 現有三個將軍，其中一個是叛徒
 - 兩種消息：攻擊、撤退
 - 深色的是叛徒
- 副官1在兩種情形下收到的消息是一樣的，無法判斷誰是叛徒
 - 如果確保左圖副官1做出正確選擇撤退，那麼在右圖中會作出錯誤選擇，即和忠誠將軍的指令不一致
 - 如果確保右圖副官1作出正確選擇，在左圖則會和忠誠的副官2做出不一致的選擇





口頭消息下的不可能性

- 一般情況下，將軍與將軍之間基于口頭消息傳播信息時
 - 在有 m 個叛徒時，少于 $3m+1$ 個參與者（包含叛徒），問題無解
- 通過反證法證明：
 - 假設共有 $3m$ 個將軍，其中 m 個是叛徒
 - 將問題簡化為3將軍問題
- $3m$ 個將軍有解 等價于 3將軍問題有解





口頭消息(OM)協議

結論： 假設將軍個數為 N ，且最多存在 m 個叛徒時，當 $N \geq 3m + 1$ 時，
OM(m) 算法可以確保所有忠誠的將軍達成一致

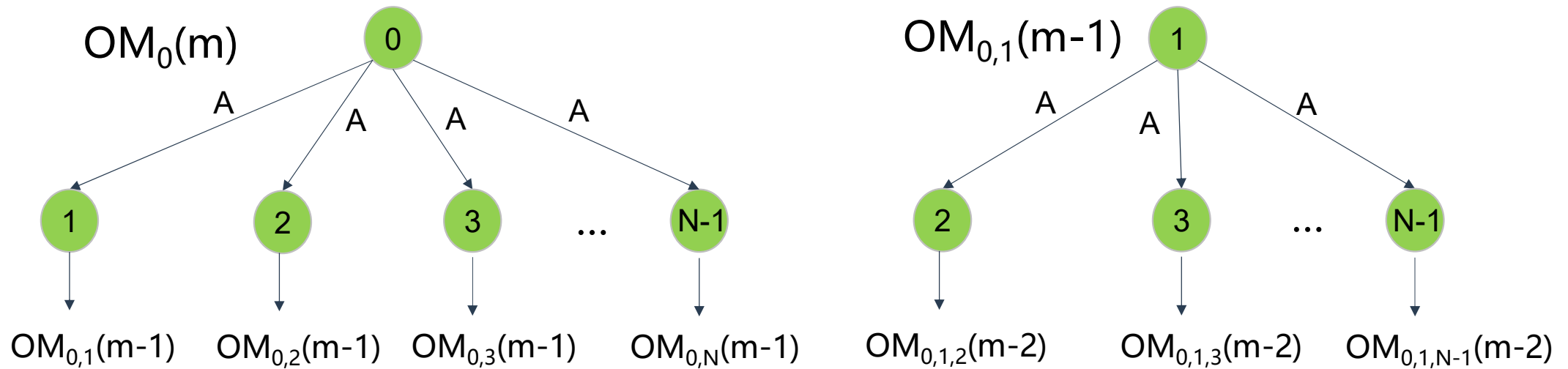
- 假設
 - 每個消息都被正確發送
 - 被發送的消息在回合結束前都能被正確接收（同步假設）
 - 消息的接收者知道誰是發送者
- 確保
 - 叛徒不能通過說謊來影響共識
 - 叛徒不能通過保持沉默來影響共識

證明思想： 根據數學歸納法完成證明，即 $m=1$ 時成立，並且當 $m=k$ 時成立，
能推導出 $m=k+1$ 時也成立

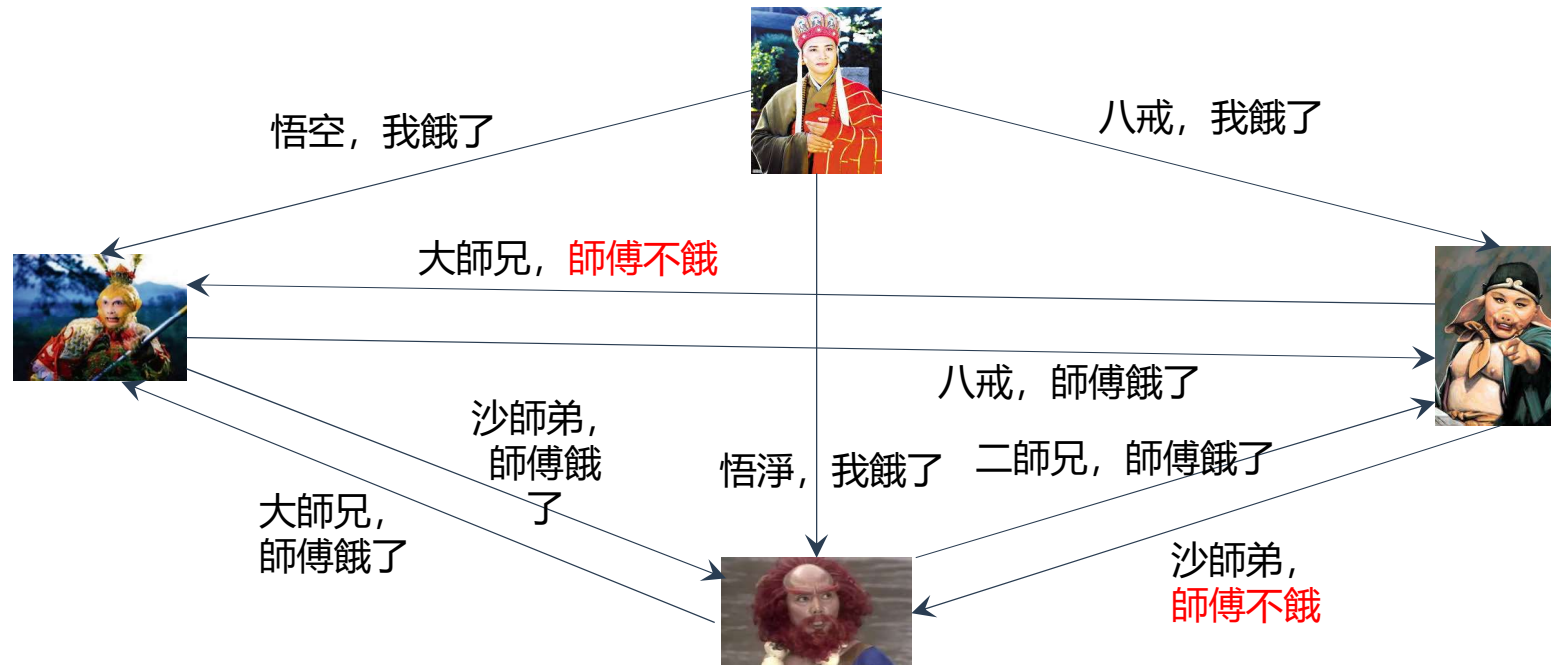


口頭消息協議

- 假設 N 個副官編號分別為 $0, 1, \dots, N-1$ ，初始狀態下 0 為將軍，并執行 $OM_0(m)$
- $OM_0(m)$ 的運流程為：將軍 0 給其它副官發送指令，然後其它副官分別執行 $OM_{0,i}(m-1)$ ，將從將軍處收到的指令發給剩餘副官
- $OM_{k_1, k_2, \dots, k_v, i}(m-v)$ 表示的 OM 算法是在除去副官 $k_1, k_2, \dots, k_v$ 後剩餘的副官中執行，且副官 i 將從副官 k_v 執行 $OM_{k_1, k_2, \dots, k_v}(m-v+1)$ 時發送的指令發給剩餘副官

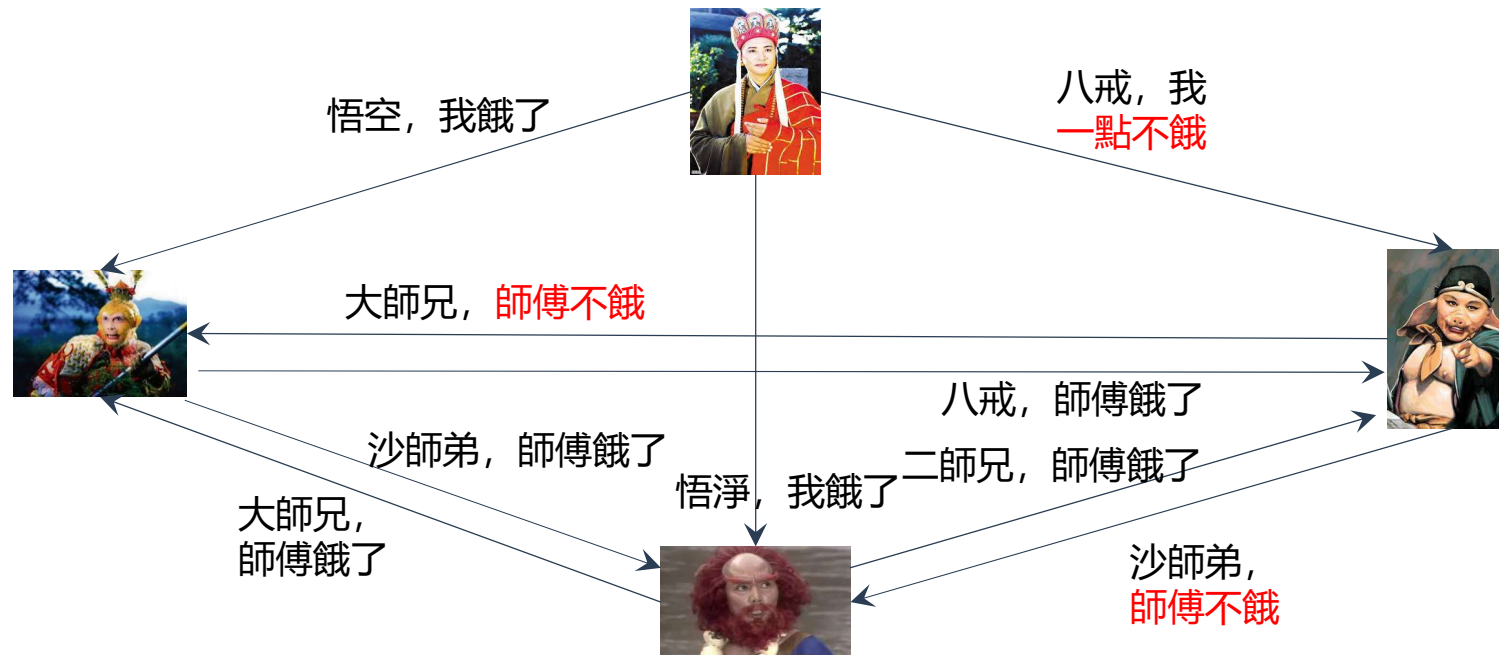


四將軍問題實例：懶惰的八戒



- 悟空和沙僧都收到了（餓了，餓了，不餓），最後都認定師傅餓了
- 滿足了IC1，師傅、悟空、沙僧的意見是一致的
- 滿足了IC2，悟空和沙僧都聽師傅的

四將軍問題實例：機智的師傅

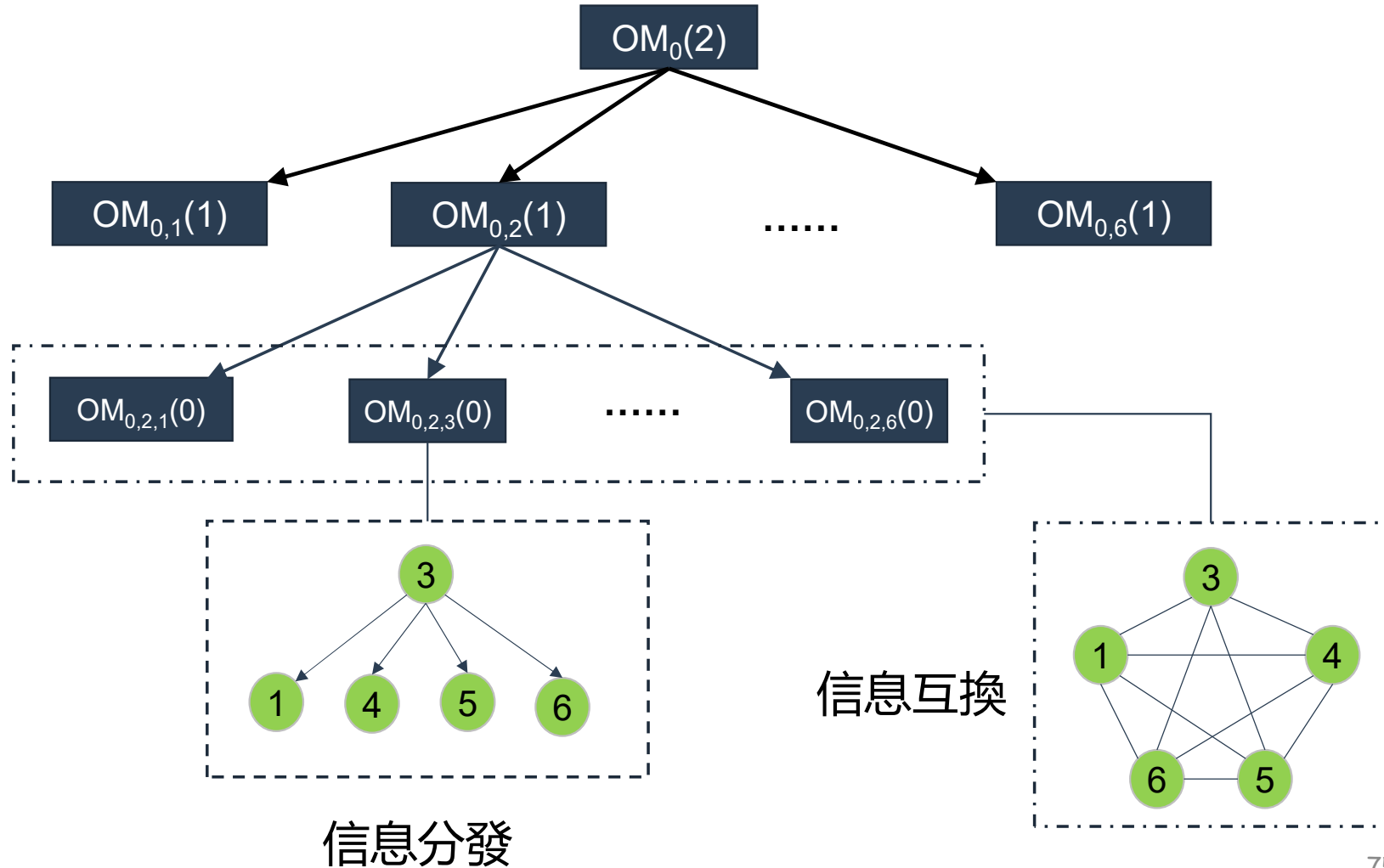


- 悟空、沙僧、八戒都收到了（餓了，餓了，不餓），都認定師傅餓了
- 滿足了IC1，悟空、沙僧、八戒的意見是一致的
- 無需滿足了IC2，師傅撒謊了



7將軍2叛徒算法分析

- OM(2): 將軍0將消息發送給副官1~6
- OM(2) 算法是爲了0~6之間的一致
- OM(1) 本質上是某節點完成一次信息的廣播後，剩餘節點對收到的消息進行互換
- OM(0) 本質上是完成一次信息的廣播

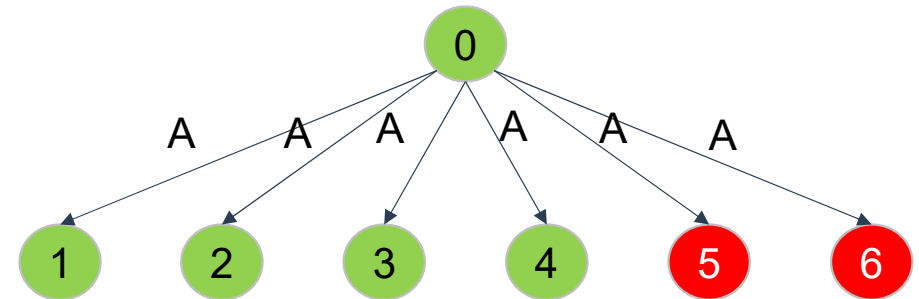




場景一：將軍是忠誠的

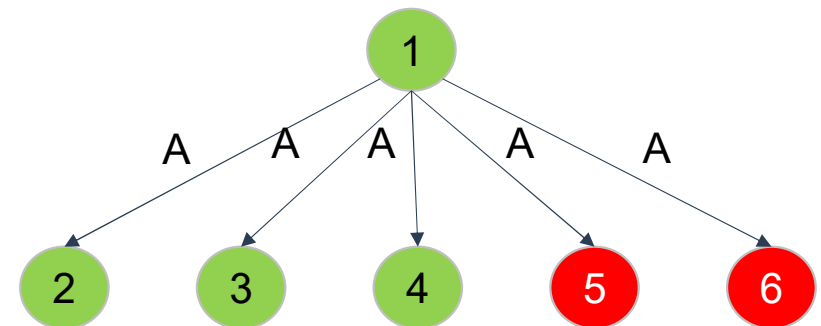
- $OM_0(2)$ 分發消息(A:attack, R:retreat)

接收 發送	1	2	3	4	5	6
0	A	A	A	A	A	A



- 每個節點都執行一次 $OM_{0,i}(1)$ ，以1執行 $OM_{0,1}(1)$ 為例：為了達成對1的判斷，2~6間交換信息

接收 發送	2	3	4	5	6
1	A	A	A	A	A

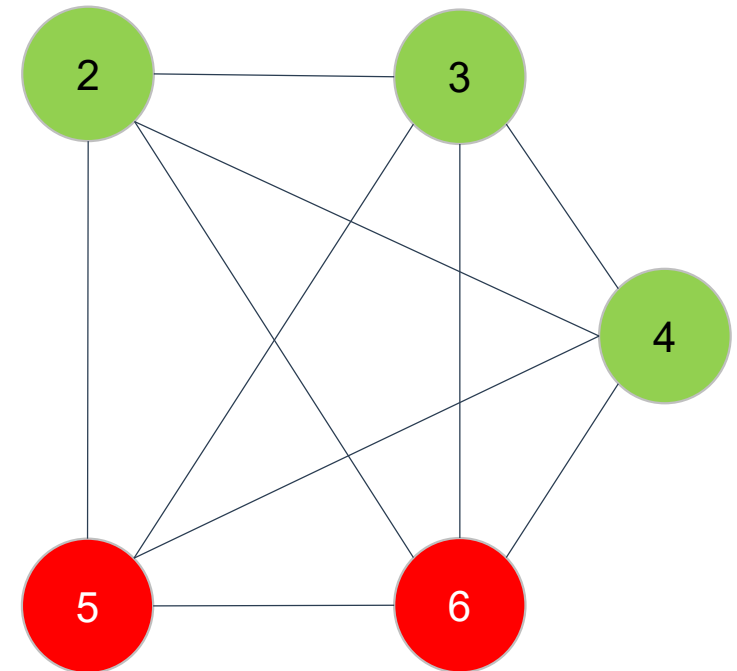




場景一：將軍是忠誠的

- $OM_{0,1,i}(0)$: 爲了達成對1的判斷, 2~6間交換信息

接收 發送	2(A)	3(A)	4(A)	5	6
2		A	A	A	A
3	A		A	A	A
4	A	A		A	A
5	x2	x3	x4		x6
6	y2	y3	y4	y5	
決策	A	A	A	a	b

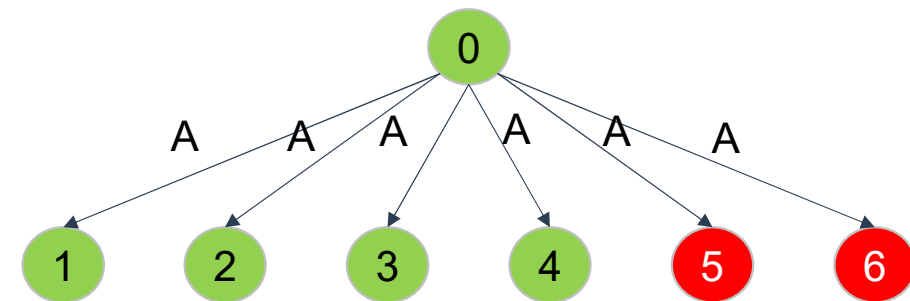
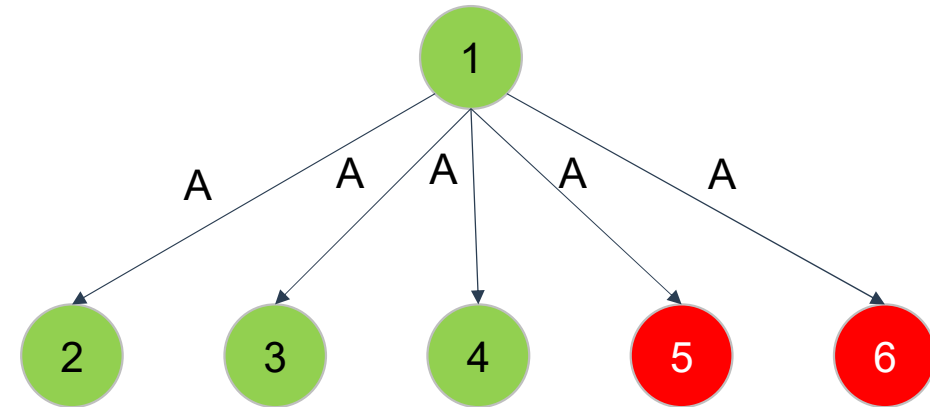


- 無論5和6發送什麼消息, 2/3/4得到的信息序列都爲(A, A, A, x, y)形式的



場景一：將軍是忠誠的

- 回到 $OM_{0,1}(1)$ 中，由于副官1發送的指令為A，所以2、3、4的最終序列為(A, A, A, A, x, y)，故決策為A，與忠誠的副官1一樣
- 回到 $OM_0(2)$ ，可得到副官1~4在分別執行 $OM_{0,i}(1)$ 時，每個副官都能獲得決策A
- 因此，6個 $OM_{0,i}(1)$ 執行完後，副官1~4得到的決策序列都為(A, A, A, A, A, x, y)；故最終決策為A，與將軍0一樣

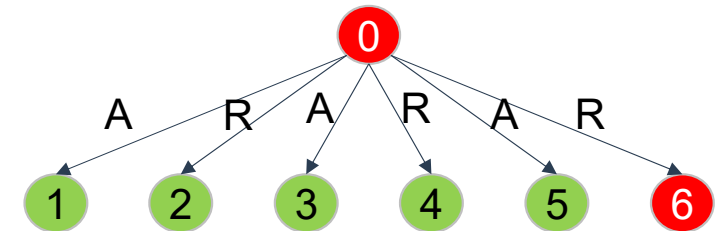




場景二：將軍是叛徒

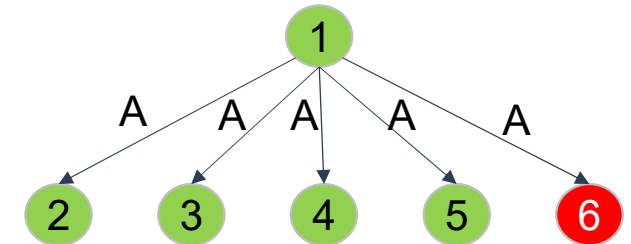
- $OM_0(2)$ 分發消息(A:attack, R:retreat)

接收 發送	1	2	3	4	5	6
0	A	R	A	R	A	R



- $OM_{0,1}(1)$ 以副官1作為將軍發送消息為例

接收 發送	2	3	4	5	6
1	A	A	A	A	A



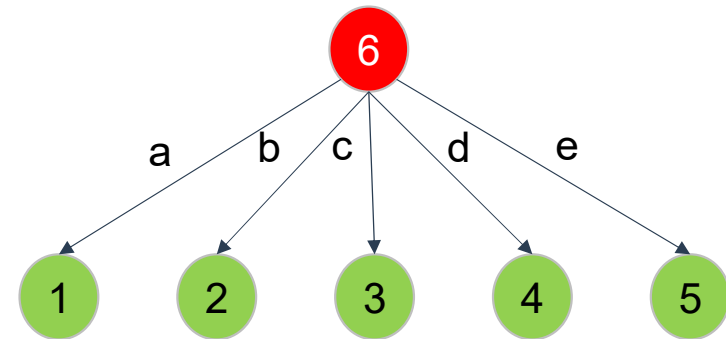
- $OM_{0,1,i}(0)$ 中2~6交換信息，顯然最終2~5會認定1的決策為A
- 故副官1的 $OM_{0,1}(1)$ 過程1~5對於副官1的決策為A達成一致
- 同理，可得2,3,4,5在 $OM_{0,i}(1)$ 中的決策結果分別為R, A, R, A（由於他們是誠實的，所以同步的結果與將軍發送的一致）



場景二：將軍是叛徒

- 在調用完 $OM_{0,i}(1)$ 之後，1~5可以得到如下決策， x_i 是副官 i 在 $OM_{0,6}(1)$ 中從副官6處收到的值

1	2	3	4	5	6
A	R	A	R	A	x_i



- 此時只需說明 $x_1 = x_2 = x_3 = x_4 = x_5 = C$ 即可得到忠誠副官所得到的決策向量都為 (A, R, A, R, A, C) ，由此可得忠誠副官的行為一致
- 此時無論 $a \sim e$ 為何值，由于1~5為忠誠的，經過 $OM_{0,6,i}(0)$ 交換信息之後，他們得到的向量均為 (a, b, c, d, e) ，故他們對於副官6的決策是一致的



課後思考

思考：經分析， $m=1,2$ 時，當 $N \geq 3m+1$ 時，OM(m) 算法可以確保所有忠誠的將軍達成一致；現假設 $m=k$ 時成立，證明 $m=k+1$ 時也成立

- 提示：可分兩步進行
 - 先證明當將軍為忠臣時，且最多存在 k 個叛徒時，當 $N > 2k+m$ 時， OM(m) 算法可以確保所有忠誠的將軍達成一致
 - 再證明當將軍為叛徒時， $N \geq 3m+1$ 成立



口頭消息的缺點

- 開銷巨大
 - $OM(m)$ 調用 $n - 1$ 次 $OM(m-1)$
 - $OM(m-1)$ 調用 $n - 2$ 次 $OM(m-2)$
 - $OM(m-2)$ 調用 $n - 3$ 次 $OM(m-3)$
 -
 - $OM(m-k)$ 被調用 $n-1, n-2, \dots, n-k$ 次
 - $O(n^m)$
- 口頭消息不能溯源（無法確定消息是否被篡改），導致需要傳輸大量的信息



方案二：簽名消息(SM)協議

改進：在實際系統中，可以引入簽名機制確保節點發送的消息無法被篡改，從而得到簽名消息算法SM(Signed Messages)

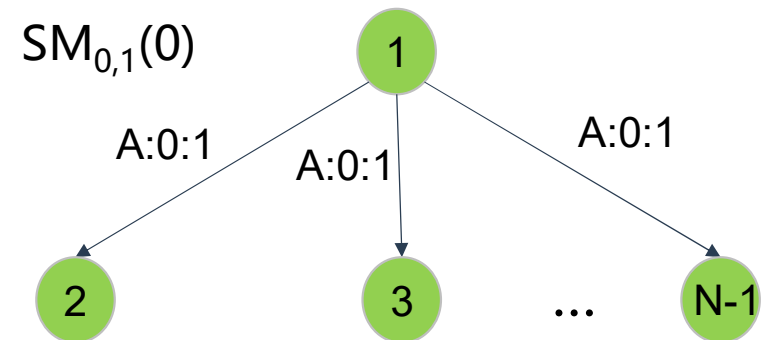
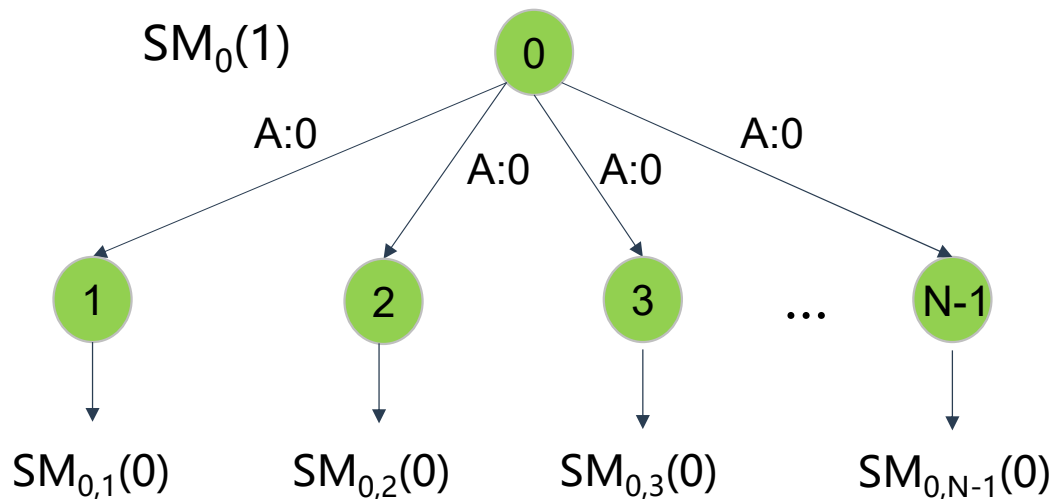
- 在口頭消息協議的基礎上添加假設
 - 忠誠將軍的簽名不可偽造
 - 任何人都可以驗證將軍的簽名
- 確保
 - 叛徒不能通過說謊來影響共識
 - 叛徒不能通過保持沉默來影響共識

結論：假設將軍個數為 N ，且最多存在 m 個叛徒時，當 $N \geq 2m + 1$ 時，SM(1) 算法可以確保所有忠誠的將軍達成一致



簽名消息協議

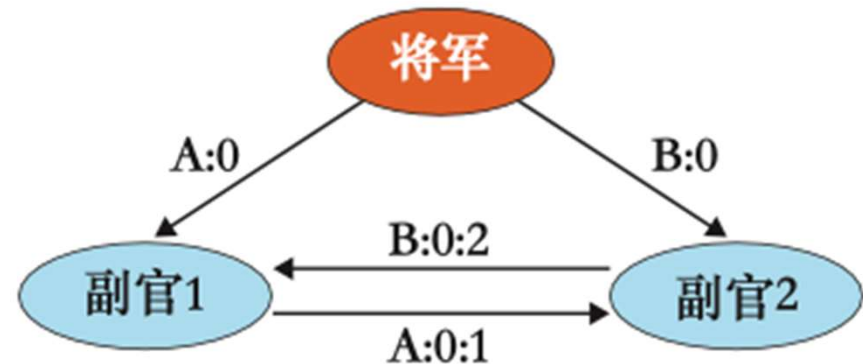
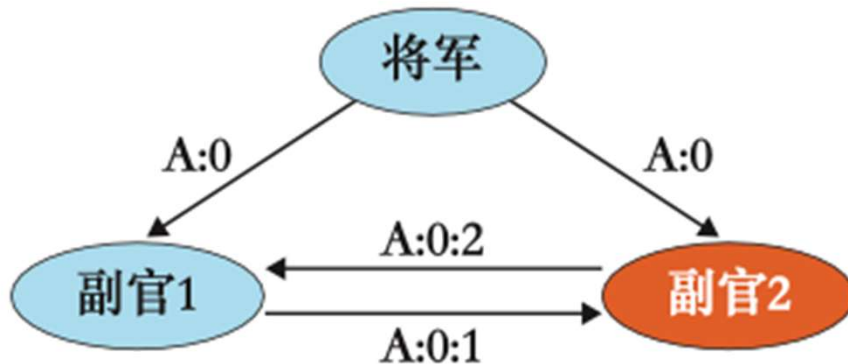
- 假設 N 個副官編號分別為 $0, 1, \dots, N-1$ ，初始狀態下 0 為將軍，並執行 $SM_0(1)$ ，從而給其它副官發送附帶簽名的指令 $x:0$
- 收到 $x:0$ 後，副官 i 執行 $SM_{0,i}(0)$ ，即在簽名指令 $x:0$ 上附帶自己簽名得到 $x:0:i$ ，然後將其發送給其它副官
- 每個副官收集來自所有副官的指令（包括自己和將軍在內），當出現不同指令時則撤退（說明將軍是叛徒）；否則當超過半數為相同指令時選擇該指令





實例分析: 3將軍1叛徒

- 如果將軍是忠誠的，副官無法偽造信息，因此副官2只能發送將軍的原指令；且無論副官2是否發送指令，副官1都將收到兩個指令A（自己和將軍的），然後執行A，和將軍一樣；
- 如果將軍是叛徒，若將軍發送不同指令，副官通信之後，收到的消息都為{A:0, B:0}，驗證簽名後則發現將軍叛變了，副官最終選擇撤退





2m+1將軍m叛徒分析

- 與3將軍類似，若將軍不是叛徒，則所有忠誠的副官都能收到超過半數與將軍相同的指令
- 若將軍為叛徒，其發送不同指令時會被發現，因此只能發送相同指令

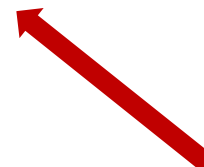
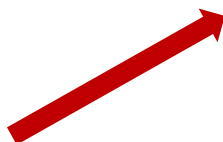
問題：若將軍是叛徒且不發送指令怎麼辦？

解決思路：副官們可以約定一個時間，若超過該時間還未收到將軍的指令，則選擇廢除將軍，並選擇新的將軍。該過程在實際系統中也稱為Leader更換過程，在簽名消息協議的基礎上進行改進，並結合Leader更換協議，就能獲得現階段實際使用的**拜占庭容錯共識**



拜占庭容錯共識的應用場景：區塊鏈

區塊鏈：構造一個對客戶端來說可信的分布式賬本；容忍少數拜占庭節點作惡
解決分布式系統不同節點間協作時的信任問題

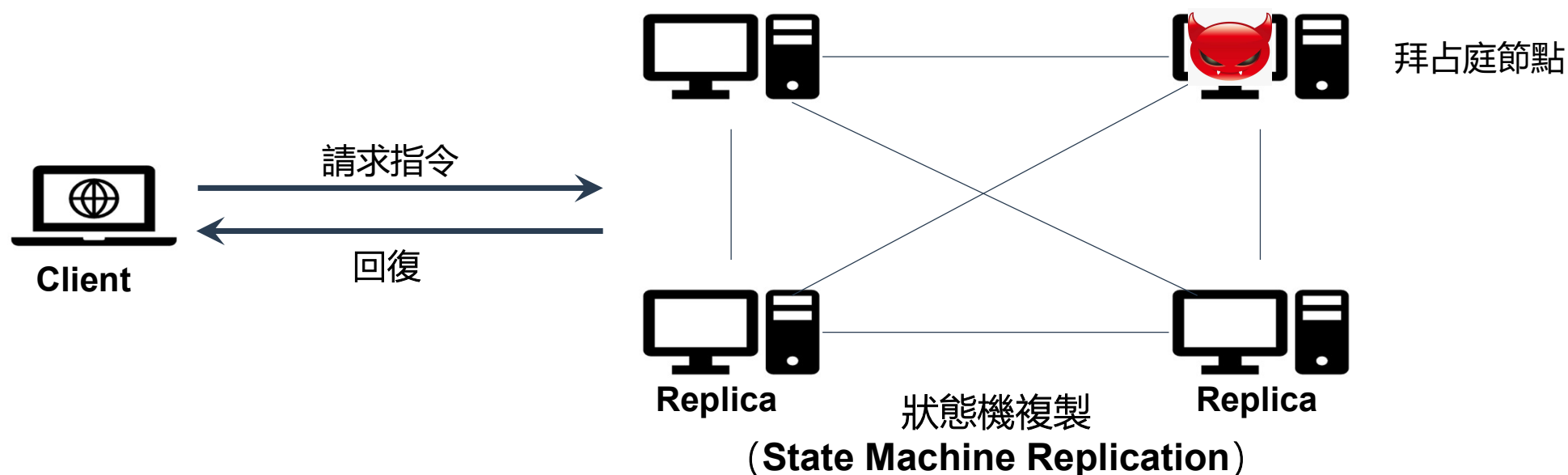


FISCO BCOS 应用领域





實際模型—狀態機複製



- **安全性 (Safety)**：對於客戶端的所有請求，服務端任意兩個正確節點都能獲得一致的排序結果并執行 (something “bad” will never happen)
- **活性 (liveness)**：服務端最終會就結果達成一致，因此對於任意客戶端的請求，服務端最終總能產生正確的答覆 (something “good” will must happen (but we don’ t know when))



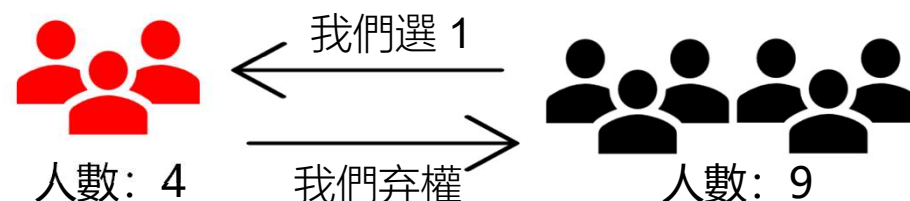
拜占庭共識：Quorum設計準則

基本規則：分布式系統每階段提案中，誠實節點最多只對一個提案投一次票



$q=8$ ❌

$q=9$ ✅



$q=10$ ❌

$q=9$ ✅

- 目的1-安全性：Quorum值 q 應該足夠大，避免通過兩個**矛盾提案**

解決方案：針對兩個提案可能產生的最高總投票數 $\leq 2*q - 1$

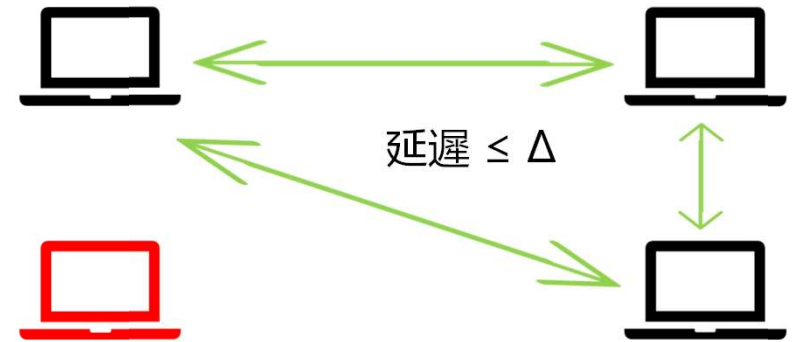
- 目的2-活性：Quorum值 q 應該足夠小，避免錯誤節點不投票使提案**無法通過**

解決方案： $q \leq$ 最低可能產生的投票數； $q >$ 拜占庭節點個數



同步BFT共識中的Quorum設計

同步網絡模型：實際網絡延遲不超過**已知最大值 Δ** ，即網絡一直都是保持穩定狀態，通過計時機制可以確定所有其他節點都正確收到了所有消息。



爲了容忍 f 個拜占庭節點，Quorum的值 q 以及節點總數 N 的設計：

具體設計：系統中出現多個矛盾提案時能被檢測到，只存在一個提案；

最低票數爲 $N-f$ ；

邊界條件： $f+1 \leq q \leq N-f \Rightarrow N \geq 2f+1, f+1 \leq q \leq N-f$

最優解： $q = f+1, N = 2f + 1$ 。



异步BFT共識中的Quorum設計

异步網絡模型： 實際網絡延遲不存在任何延遲上限，但節點間經過無限重傳能確保消息最終被正確接收



爲了容忍 f 個拜占庭節點，Quorum的值 q 以及節點總數 N 的設計：

具體設計： 針對兩個提案可能產生的最高票數爲 $N+f$;

最低票數爲 $N-f$;

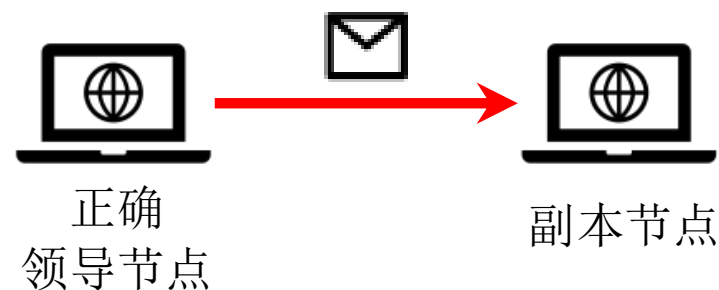
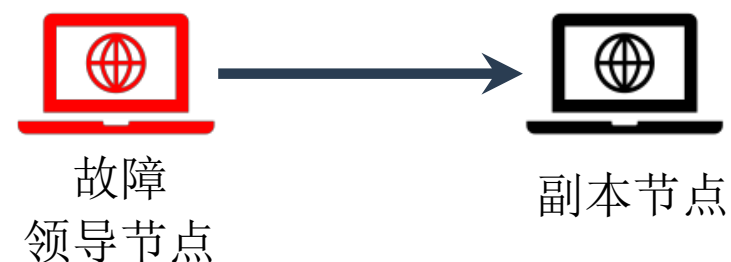
邊界條件： $f+1 \leq q \leq N-f$, $N+f \leq 2*q - 1 \Rightarrow N \geq 3f+1$, $2f+1 \leq q \leq N-f$

最優解： $q = 2f+1$, $N = 3f + 1$ 。



同步和异步BFT共識的缺陷

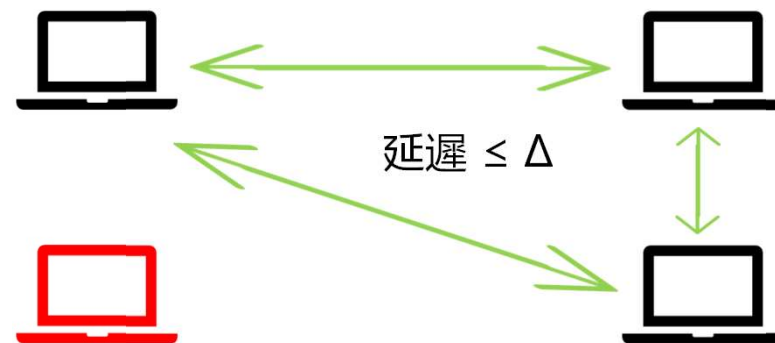
- 同步BFT共識假設網絡一直保持穩定狀態，若網絡出現不穩定現象，即延遲超過預設最大值，則會出現安全問題
- 异步BFT共識只能采用**無Leader的共識模式**，因為异步網絡中無法分辨Leader錯誤和網絡延遲過高兩種情況；因此，异步BFT共識不僅交互輪數過多，還可能導致理論上的活鎖（類似于前面分析的Paxos共識）





引入部分同步網絡模型

部分同步網絡模型：網絡狀態不可預測，但總存在一個網絡滿足同步假設的時間窗口，此時共識能穩定運行；網絡异步時，則專注于確保共識的安全性



- 部分同步BFT共識的典型案列
 - PBFT：第一個實用化的BFT共識機制；基于三階段共識實現節點間狀態一致；采用數字簽名機制實現投票
 - Hotstuff：2019年被提出，是對BFT共識的一次重大改進；基于四階段共識實現節點間狀態一致，降低leader更換開銷；采用門限簽名機制實現投票，并采用流水綫機制減少交互信息量



實用拜占庭容錯-PBFT



Barbara Liskov

美國杰出計算機科學家，麻省理工學院電子電氣與計算機科學系教授

2004年約翰·馮諾依曼獎得主，**2008年圖靈獎得主**

美國第一位計算機科學女博士，歷史上第二位獲得圖靈獎的女性

- Practical Byzantine Fault Tolerance, OSDI 1999
- 狀態機複製系統
- 目標：即使在少部分服務器為拜占庭節點的情況下，使來自客戶端的請求在每個服務器上都按照一個確定的順序執行
- 適用於有強一致性要求的私有鏈和聯盟鏈場景，IBM超級賬本可選共識協議



PBFT的主要思想

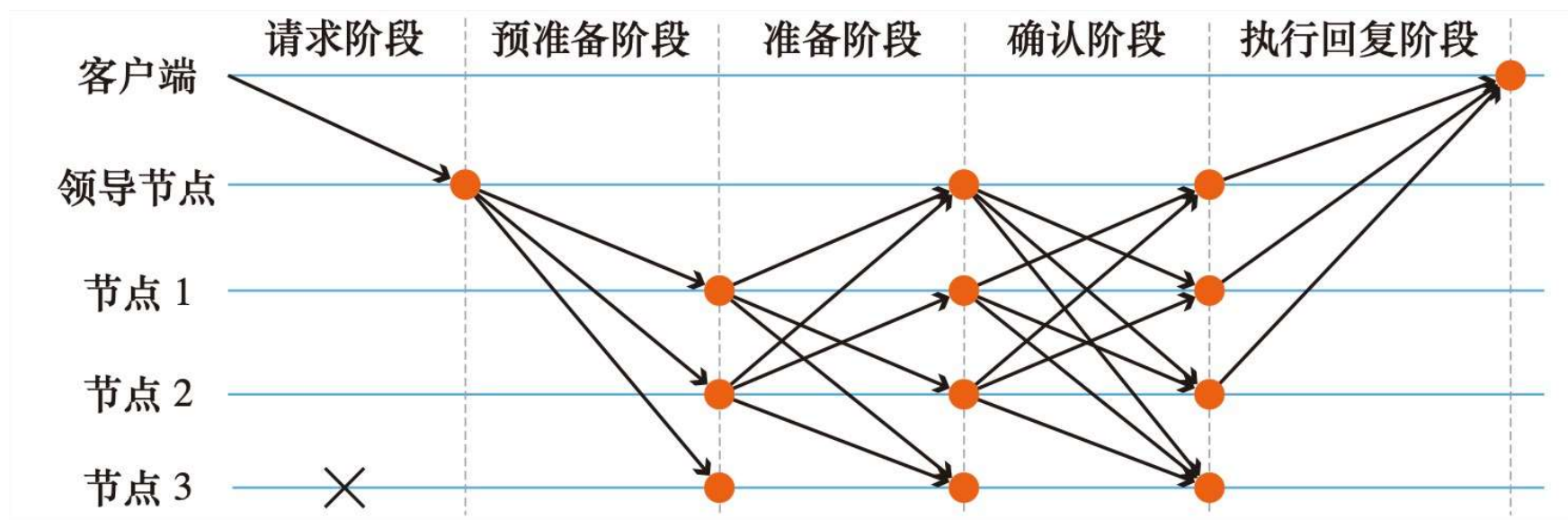
**PBFT基于Leader完成共識過程，并基于Quorum機制確保安全性和活性；
PBFT采用部分同步假設，假設故障節點個數不超過 m 個，總節點 $3m+1$ 個，
Quorum設計 $q=2m+1$**

- PBFT在Paxos、Raft等故障容錯算法的二階段共識的基礎上增加了一個階段
- 二階段共識後的交易處于準備完成狀態，此時，節點廣播信息告訴其它所有節點自己已完成準備
- 當節點收到 q 個準備完成的投票時提交并執行該交易
- 所有節點都對Leader節點進行監測，若長時間未收到Leader發送的消息，則觸發視圖變更協議，更換Leader節點



4節點實例分析

- Leader節點收到客戶端請求後，為其分配序號，然後發送給其它節點
- 其它節點收到預準備信息後，廣播一個準備信息
- 當節點收到3個準備信息，則完成準備，然後廣播提交信息
- 當收到3個提交信息後，則執行相應的請求指令，并回復客戶端





Hotstuff

HotStuff: BFT Consensus with Linearity and Responsiveness

——PODC 2019

- 目標：實現具備綫性複雜度的拜占庭共識

HotStuff與libra的關係：

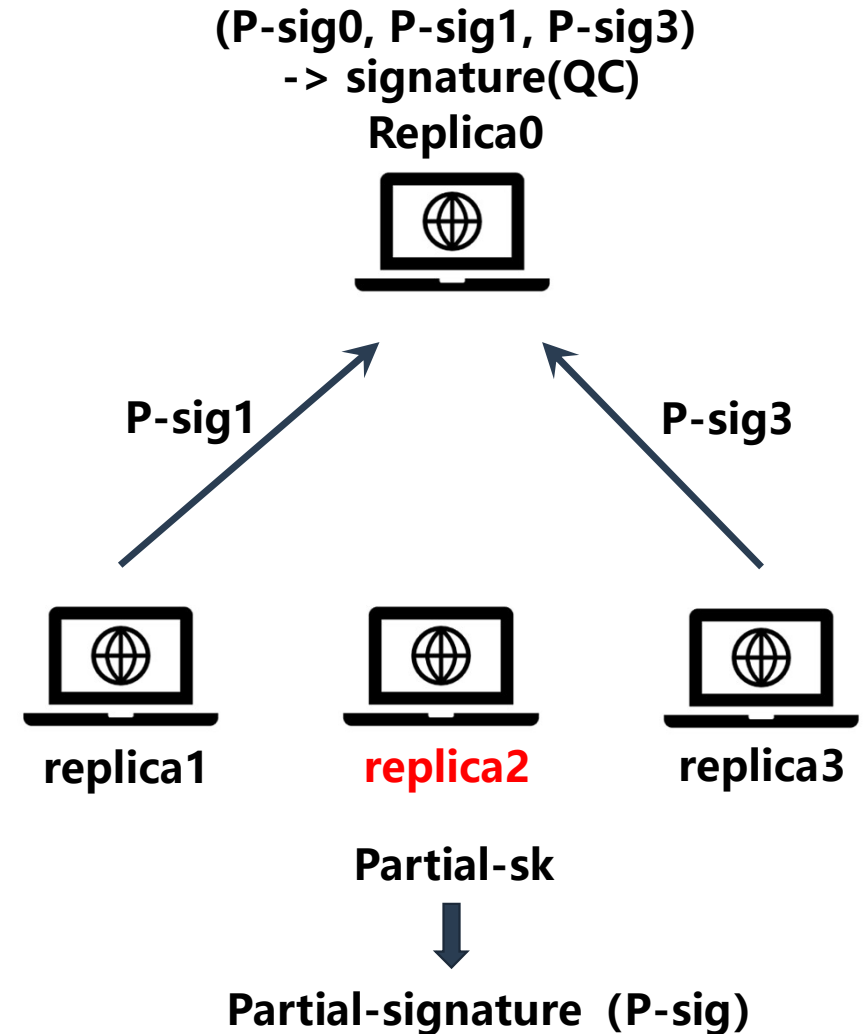
Facebook在2019年發布了libra
白皮書，裏面提到的LibraBFT共
識是基于HotStuff演化而來





協議基礎

- **門限簽名**: 在 (k, n) 門限簽名中, 每個節點擁有部分私鑰 Partial-sk , 可進行部分簽名, 任意 k 個節點的部分簽名可以合成一個完整簽名, 幷能被全域公鑰驗證
- **View**: 共識視圖, 每個視圖由不同 leader 節點引導完成一輪共識
- **投票**: 節點對提案進行部分簽名
- **QC(Quorum Certificate)**: 由 $2m+1$ 個不同節點的部分簽名合成的一個全域簽名, 可通過全域公鑰驗證幷證明 $2m+1$ 個節點進行了投票

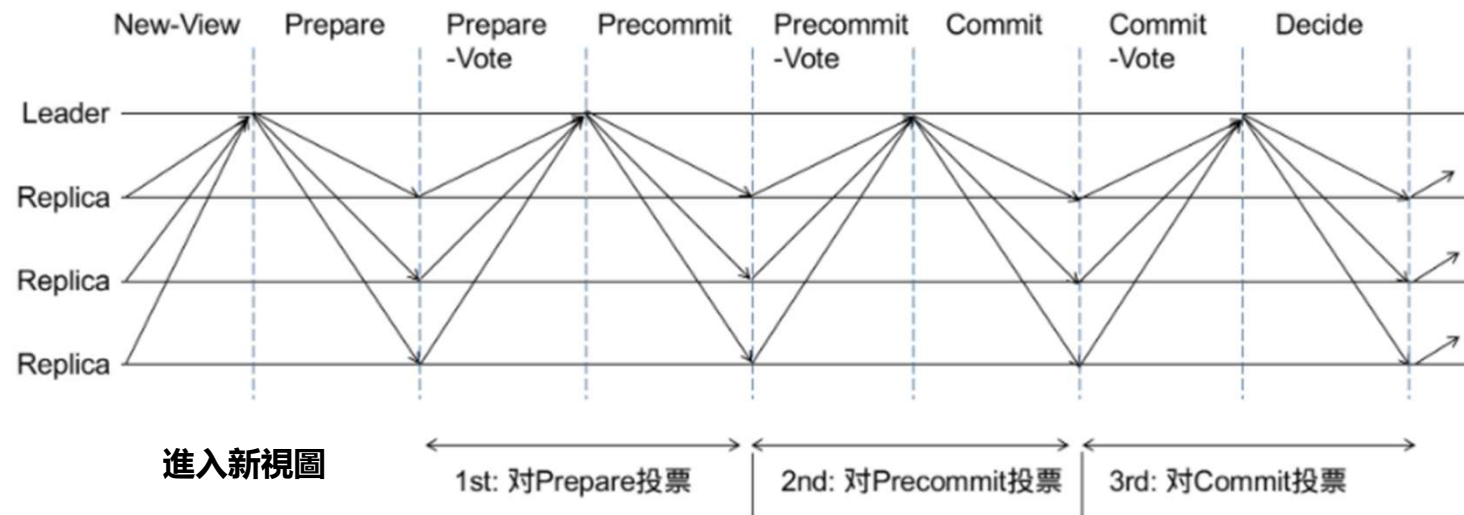




Hotstuff (基本方案)

通過門限簽名把PBFT的流程改成了四階段算法（去除了客戶請求和回復客戶兩個階段），存在以下特點：

- 每次交互只在Leader和其他節點間進行，無需所有節點都進行廣播，減少通信複雜度 ($O(n^2) \rightarrow O(n)$)
- 每個階段都是**同構**的

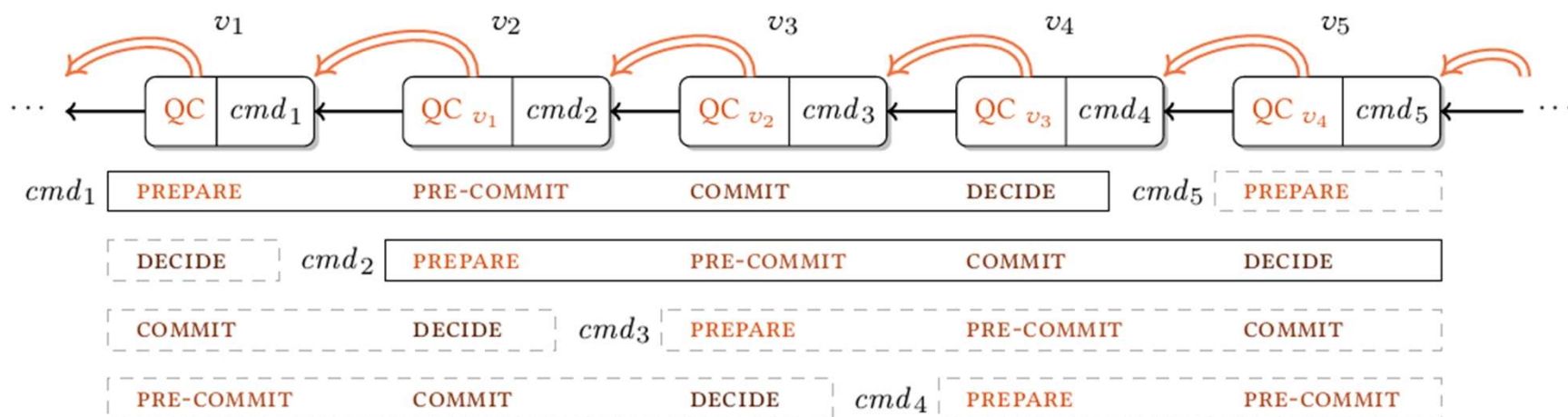




Hotstuff (流水綫方案)

基本方案存在的問題：四階段過程中，每個節點都需要廣播準備、預提交、提交三次信息

解決方案：每個區塊都引用上一個區塊，并只進行一個階段的投票。當區塊後面鏈接三個區塊時，相當于完成了四階段共識，處于可執行狀態

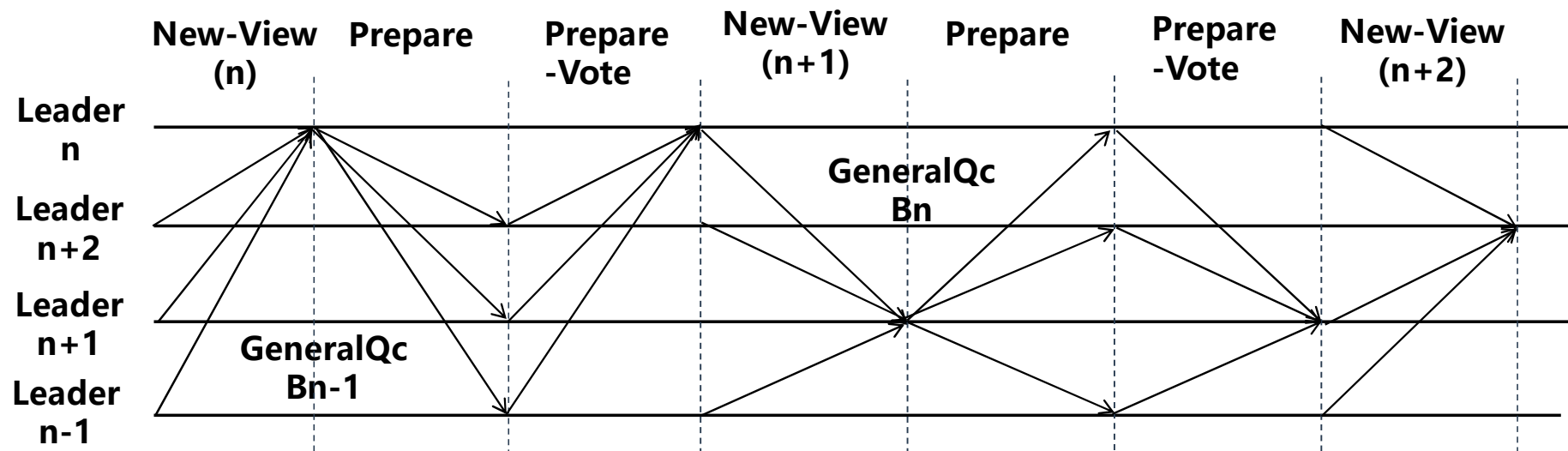




Hotstuff (流水綫方案)

一階段協議，分爲三個步驟：

- 所有節點對新的Leader投票，舊Leader把獲得的全域簽名發給新Leader
- 新Leader將上一視圖的全域簽名和新區塊發給所有有節點
- 節點對新區塊投票并發給Leader，Leader合成全域簽名





第4節 總結





總結

剖析分布式系統安全問題的根源，探討如何從交互網絡、分布式算法和信任三方面構建安全、穩定、可信的分布式系統



第一節

建立安全、穩定的交互網絡

- 交互網絡的組成
- 交互信道安全
- 彈性覆蓋網絡
- 防禦日蝕攻擊

瞭解交互網絡的組成以及如何構建安全的交互網絡



第二節

分布式算法

- 協作過程中的安全隱患
- 時鐘同步算法
- 并發問題
- 故障容錯的一致性算法

挖掘分布式系統各組件之間如何進行協同



第三節

信任問題

- 訪問控制
- 信任評價模型
- 拜占庭容錯

瞭解分布式系統中的信任問題及解決方案



展望

新的應用場景對分布式系統的研究提出了新的挑戰



- 隨著比特幣的出現，**區塊鏈**技術得到飛速發展，但是如何進一步提升區塊鏈性能并拓展其應用範圍對分布式系統研究提供了新的方向
- 近些年來，爲了打破數據孤島，**聯邦學習**被提出并得到發展，如何協調互不信任的組織之間實現可信數據協同成爲新的挑戰
- 互聯網的發展**暴露了傳統設計的弊端**，如何構建可信互聯網基礎設施至關重要